

APPRENDIMENTO AUTOMATICO

Alessandro Zappatore

Università degli studi di Torino
Anno accademico 2025/2026, 1° semestre

1 Introduzione

1.1 Definizione di Machine Learning (ML)

Thomas Mitchell; 1996; Machine Learning

A computer program is said to learn from experience with respect to some class of tasks and performance measure, if its performance at tasks in, as measured by, improves with experience.

È la combinazione di tre oggetti l'**esperienza E** sotto forma di esempi del problema risolto al computer, un **task T** da risolvere (classificazione degli esempi in un certo numero di categorie, assegnazione di un numero associato agli esempi, riconoscimento di un'immagine), sotto una **misura di performance P** un valore per misurare quanto sta andando bene.

Il programma del computer fa apprendimento automatico basandosi sugli esempi se le sue performance migliorano man mano che vede gli esempi.

Esempio del melone Assumendo di avere dei dati sulla maturazione dei meloni vogliamo sapere se il frutto è maturo o no.

ID	Color	Root	Sound	Ripe
1	green	curly	muffled	true
2	dark	curly	muffled	true
3	green	straight	crisp	false
4	dark	slightly curly	dull	false

Ora troviamo un melone con le seguenti caratteristiche e vogliamo sapere se è maturo:

Color	Root	Sound
green	curly	dull

In questo caso abbiamo un **task**, sapere se è maturo, delle **performance**, quanto è accurata la nostra predizione e dell'**esperienza** i meloni comprati in precedenza.

Questo tipo di lavoro è chiamato **apprendimento supervisionato**, dove un essere umano andrà a segnare qual è la risposta giusta. Nell'**apprendimento non supervisionato** la colonna delle risposte non è presente, ma si andrà a raggruppare risposte simili.

1.2 Terminologia

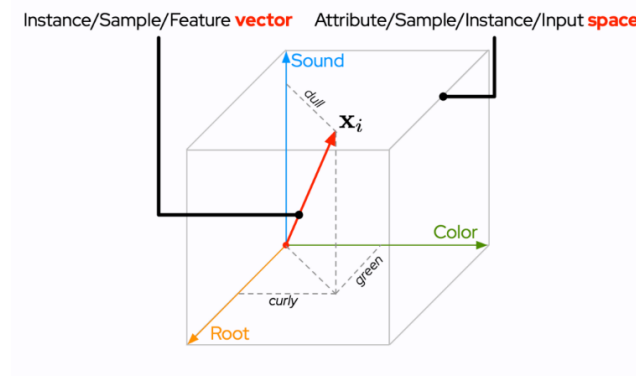
Attributes/Features				Label/Class y	
	ID	Color	Root	Sound	Ripe
Training set	1	green	curly	muffled	true
	2	dark	curly	muffled	true
	3	green	straight	crisp	false
	4	dark	slightly curly	dull	false
Test set	1	green	curly	dull	?
	Sample/Instance $\mathbf{x}_i$				
Attribute value/Feature value x_{i1}					

Le colonne della tabella sono chiamate **attributi** o **features** dell'esempio, l'elemento che ci dice come risolvere il task è chiamato **etichetta** o **classe** dell'esempio, tipicamente la classe viene denotata con una y_i minuscola.

Una riga della tabella, senza considerare la colonna della classe, è chiamato **sample** o **istanza** dell'esempio, in genere denotata con una X_i per dire che è l'iesimo esempio. [Tutte le volte che parleremo di vettori saranno rappresentati da lettere maiuscole, invece i valori scalari saranno rappresentati in corsivo]

Il singolo valore di una colonna è chiamato **attribute value** o **feature value** ed è rappresentato da un valore in corsivo x_{i1} in questo caso la prima feature dell'iesimo esempio.

L'insieme degli esempi che usiamo per indurre la regola che poi utilizzeremo per classificare è chiamato **set di training** e spesso al fianco si trova un altro insieme di esempio, chiamato **test set**, che serve per riuscire a capire quanto bene stiamo performando.



Quanto gli esempi vengono immaginati in uno spazio euclideo di qualche tipo, in questo caso a tre dimensioni, spesso ci si riferisce all'esempio come **instance vector** oppure **sample vector**, **feature vector**, l'intero insieme di tutti i possibili esempi è chiamato anche **attribute space**, **sample space**, **instance space**, **input space**.

1.3 Notazione

Symbol	Meaning
$\mathcal{X}$	instance space, in many cases $\mathcal{X} = \mathbb{R}^d$
$\mathcal{Y}$	label space, e.g., $\mathcal{Y} = \{0, 1\}$ for binary classification
$\mathbf{x}_i \in \mathcal{X}$	i -th instance/sample, $\mathbf{x}_i = [x_{i1}, x_{i2}, \dots, x_{id}]^T$
d	dimensionality of the instance space
$y_i \in \mathcal{Y}$	label of the i -th instance, e.g., $y_i = 1$ if the watermelon is ripe, $y_i = 0$ otherwise
D	Dataset, $D = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_n, y_n)\}$
n	number of instances in the dataset
h	model/hypothesis, a function $h: \mathcal{X} \rightarrow \mathcal{Y}$ that maps instances to labels
$\mathcal{L}$	Learning algorithm $\mathcal{L}: \mathcal{P}(\mathcal{X} \times \mathcal{Y}) \rightarrow \mathcal{H}$, where $\mathcal{P}$ is the power set and $\mathcal{H}$ is the hypothesis space
$\mathbb{I}$	Indicator function, $\mathbb{I}(P)$ (sometimes denoted as $\mathbb{I}[P]$) is 1 if P is true, 0 otherwise

1.4 Altra terminologia

Il **training** è un processo utilizzato da algoritmo di machine learning per costruire un modello dai dati. I dati utilizzati durante questo processo sono chiamati **training data**, dove ogni esempio è chiamato **training example** e l'insieme di tutti gli esempi è chiamato **training set**.

Obiettivo del Machine Learning

L'obiettivo dell'apprendimento automatico è trovare un **ipotesi** che è capace di approssimare la **ground-truth** su dati che non ha mai visto.

1.5 Tasks

I più importanti tasks che si possono fare con l'apprendimento automatico sono:

- **Predittivi**: predire un numero o una caratteristica, solitamente sono supervisionati
 - **Classificazione**, abbiamo un certo numero di etichette negli esempi, se le etichette sono solo due si parla di classificazione **binaria** invece se sono di più si parla di classificazione **multiclasse**.
 - **Regressione**, è un caso di predizione dove l'insieme delle etichette corrisponde con un intervallo continuo, cerchiamo di predire dei numeri.
- **Descrittivi**: descrivere qualcosa su come è strutturato il data set, solitamente non supervisionati
 - **Clustering**
 - **Regole di associazione**

Learning supervisionato l'etichetta è fornita dall'utente

Learning non supervisionato l'etichetta non esiste

1.6 Assunzione i.i.d

Nella maggioranza dei casi, in particolare in tutti gli algoritmi che vedremo nel corso, si assume che i dati siano **Indipendenti e Identicamente Distribuiti**.

Nel momento in cui ho estratto un istanza con la sua etichetta il prossimo lo estraggo dalla stessa distribuzione di probabilità, perché se cambia la distribuzione non ho speranza di apprendere nulla.

Indipendenti perché fissata la distribuzione, se io pesco due esempi, l'aver estratto un esempio non ha implicazione sull'estrazione di un altro esempio. [Ex. estraggo una pallina dall'urna, poi la rimetto dentro e riestraggo. La probabilità del primo e del secondo caso sarà la medesima]

- **Indipendenza**: $P((X_i, y_i) \mid (X_1, y_1), \dots, (X_{i-1}, y_{i-1})) = P((X_i, y_i)) \quad \forall i$
- **Identicamente distribuiti**: Tutte le istanze provengono dalla stessa distribuzione sottostante. Ciò significa anche che i dati di training e i dati di test devono provenire dalla stessa distribuzione.

Esempio

Immaginando di gestire una fabbrica di di cupcake. Ogni cupcake è fatto:

- usando la **stessa ricetta** (identicamente distribuita)
- e **e cotti in uno stampo separato** senza influenzare gli altri (indipendenti)

Quindi, se assaggi 10 cupcake a caso e hanno tutti un sapore dolce, puoi presumere che anche i cupcake futuri saranno dolci, perché il tuo campione è i.i.d.

Esempio - What if

Cosa succede se:

- All'improvviso usi **due ricette diverse** → non distribuite in modo identico.
- Cuoci i cupcake nello **stesso stampo e l'impasto si mescola** → non indipendente.

1.7 Spazio delle ipotesi

1.7.1 Deduzione Vs Induzione

Deduzione vuol dire che parto da degli assiomi, ho delle regole per combinare gli assiomi e ottengo una deduzione. La deduzione è un processo **corretto**, se assumo che gli assiomi siano corretti allora qualsiasi cosa io deduca da questi assiomi deve essere vera per forza.

L'Induzione è il processo inverso, si parte da oggetti specifici, gli esempi, e cerco di dedurre una regola generale.

Il processo di Apprendimento Automatico si basa interamente sul principio dell'induzione.

Esempio

Esempio di deduzione:

- Tutti gli umani sono mortali
- Socrate è un uomo
- Quindi, Socrate è mortale

Esempio di induzione:

- Dopo aver osservato che ogni essere umano conosciuto prima o poi muore, potremmo dedurre che tutti gli esseri umani sono mortali.

1.7.2 Symbolic concept learning

Con i metodi induttivi vogliamo provare a costruire delle regole logiche, e in questo caso ci troviamo nel campo del **symbolic concept learning** e una sua forma più estesa è chiamata **Inductive Logic Programming**, dei linguaggi che ci permettono di lavorare con la logica per costruire queste regole in modo induttivo.

La forma base del concept learning è chiamato **boolean concept learning** dove l'obiettivo è costruire una funzione $h : \chi \mapsto \{0, 1\}$.

Esempio

Supponendo di voler predire se un melone è maturo e la formula è:

$$ripe \iff (color = ?) \wedge (root = ?) \wedge (sound = ?)$$

Al posto dei punti interrogativi possiamo mettere un valore oppure un **asterisco *** per dire che **non interessa il valore specifico**.

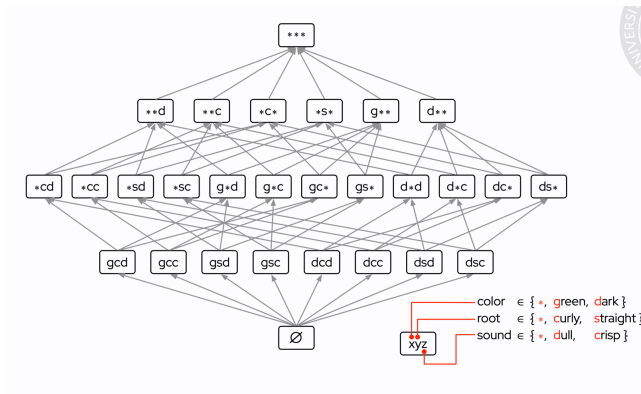
In questo caso il melone sarà maturo se il suono è dull il resto non mi interessa.

$$ripe \iff (color = *) \wedge (root = *) \wedge (sound = dull)$$

1.7.3 Spazio delle ipotesi

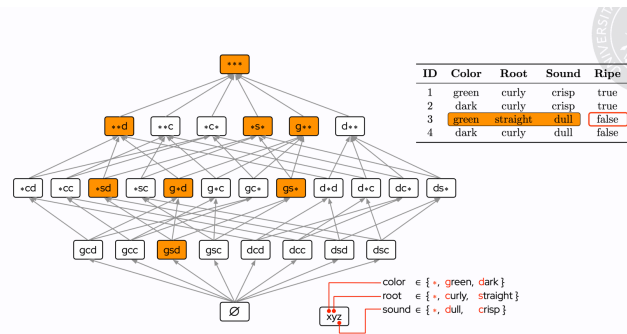
Lo **Spazio delle ipotesi** è l'insieme di tutte le ipotesi (tutti i possibili modelli) che posso apprendere utilizzando uno specifico algoritmo di apprendimento. [Nell'esempio sopra citato è l'insieme di tutte le formule booleane che posso ottenere].

Possiamo pensare all'apprendimento automatico come un **processo di ricerca all'interno dello spazio delle ipotesi**, e devo trovare l'ipotesi migliore in base ai dati che ho.

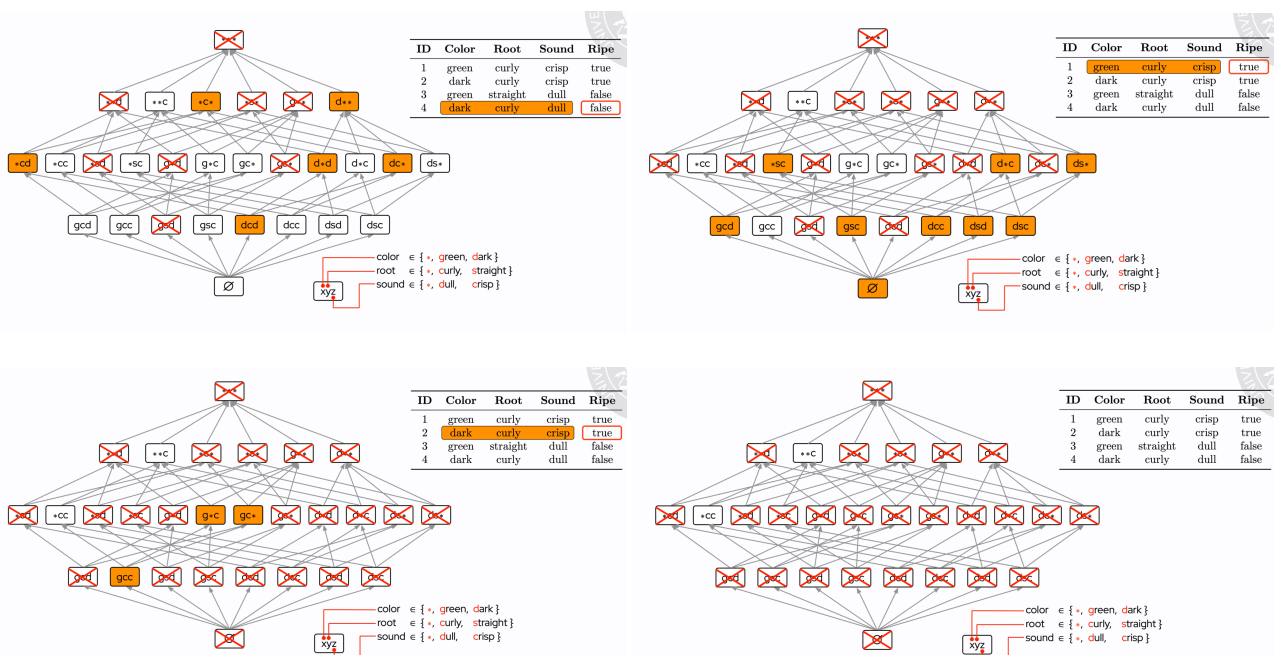


1.7.4 Spazio delle versioni

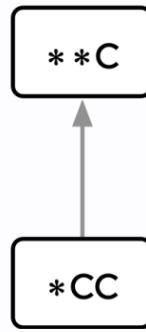
Lo **spazio delle versioni** è un **sottoinsieme dello spazio delle ipotesi** che è consistente con tutti gli **esempi di training**. Sostanzialmente prendo solo gli esempi che danno una risposta giusta [nel caso dei meloni se trovo un esempio con suono *dull* ma non maturo lo andrò a scartare].



L'ipotesi in arancione non è consistente con l'esempio, perché questo avrebbe detto a qualsiasi cosa mature. **d* dice che dovrebbe dare maturo e anche questo non va bene. Tutte le caselle evidenziate in arancione non sono consistenti con l'esempio in questione, quindi decido di eliminarle.



Vado a controllare tutti gli esempi e alla fine andrò a vedere che lo spazio delle versioni finale sarà:



Nell'esempio lo spazio delle versioni consiste in due ipotesi:

- $ripe \iff (color = *) \wedge (root = *) \wedge (sound = crisp)$
- $ripe \iff (color = *) \wedge (root = curly) \wedge (sound = crisp)$

Dato quello che sappiamo del dataset le due sono equivalenti e non c'è una giusta.

1.7.5 Bias induttivo

Nel caso dovesse arrivare un nuovo esempio del tipo $Color = green, Root = straight, Sound = crisp$, avremmo una regola che dice maturo e una che dice non maturo.

Senza altre informazioni non possiamo decidere quale delle due ipotesi è corretta, però un algoritmo non può restituire un insieme di ipotesi.

Tutti gli algoritmi di apprendimento hanno un **bias induttivo**, non basato sui dati.

Si può scegliere di preferire delle regole più generali o all'opposto potrebbe preferire una regola più specifica.

Ogni algoritmo di apprendimento DEVE avere un bias induttivo, altrimenti non potrebbe concludere l'apprendimento. Potrei anche scegliere una risposta a caso ma non sarebbe una soluzione furba da prendere.

Occam's Razor è un principio che a parità di ipotesi consistenti devo scegliere quella più semplice come bias induttivo.

Semplice può avere molti significati diversi, in genere la più utilizzata è quella che fa meno assunzioni sui dati.

No Free Lunch Theorem in modo informale dice che nessun algoritmo di apprendimento universalmente migliore di altri quando le sue performance sono misurate su tutti i possibili mondi.

1.7.6 No Free Lunch Theorem

Denotiamo con:

- $P(h \mid X, \mathcal{L}_a)$: **probabilità** che l'algoritmo $\mathcal{L}_a$ restituisca l'ipotesi h dati i dati X
 - h è l'ipotesi, quello che stiamo imparando
 - X è il training set
 - $\mathcal{L}_a$ è l'algoritmo di apprendimento
- f è la **ground truth** il modello vero sottostante che vogliamo apprendere
- l'errore **out-of-sample** dell'ipotesi h , denotato con E_{ote} , è la media dell'errore su tutti gli elementi che non sono presenti nel training set, che sono commessi dai modelli appresi tramite $\mathcal{L}_a$ sul training set X .

$$E_{ote}(\mathcal{L}_a \mid X, f) = \sum_h \sum_{x \in \chi - X} P(X) \mathbb{I}(h(X) \neq f(X)) P(h \mid X, \mathcal{L}_a)$$

Faccio la somma su tutte le possibili ipotesi, dopo di che sommo su tutti gli esempi che non sono nel training set ottenuto da $\mathcal{X}$ (spazio di tutti i possibili esempi) togliendo gli elementi del training set. $P(X)$ è la probabilità di estrarre uno specifico esempio per la funzione indicatrice $\mathbb{I}(h(X) \neq f(X))$ (restituisce 1 se l'espressione è vera e il modello sta sbagliando a predire l'etichetta di X e 0 altrimenti) tutto per la probabilità di h su tutti i modelli.

Dimostrazione

Vogliamo dimostrare che l'errore non dipende da $\mathcal{L}_a$, quindi un algoritmo vale l'altro in media.

$$\begin{aligned}\sum_f E_{ote}(\mathcal{L}_a | X, f) &= \sum_f \sum_h \sum_{\mathbf{x} \in \mathcal{X}-X} P(\mathbf{x}) \mathbb{I}(h(\mathbf{x}) \neq f(\mathbf{x})) P(h | X, \mathcal{L}_a) \\ &= \sum_{\mathbf{x} \in \mathcal{X}-X} P(\mathbf{x}) \sum_h P(h | X, \mathcal{L}_a) \sum_f \mathbb{I}(h(\mathbf{x}) \neq f(\mathbf{x})) \\ &= \sum_{\mathbf{x} \in \mathcal{X}-X} P(\mathbf{x}) \sum_h P(h | X, \mathcal{L}_a) \frac{1}{2} 2^{|\mathcal{X}|} \\ &= \frac{1}{2} 2^{|\mathcal{X}|} \sum_{\mathbf{x} \in \mathcal{X}-X} P(\mathbf{x}) \sum_h P(h | X, \mathcal{L}_a) \\ &= \frac{1}{2} 2^{|\mathcal{X}|} \sum_{\mathbf{x} \in \mathcal{X}-X} P(\mathbf{x}) \cdot 1\end{aligned}$$

Nella diapositiva precedente, il termine $\frac{1}{2} 2^{|\mathcal{X}|}$ è il risultato della somma degli errori di tutte le possibili ipotesi binarie. Dato uno spazio campionario $\mathcal{X}$, esistono $2^{|\mathcal{X}|}$ modi per etichettare gli esempi e, quindi, $2^{|\mathcal{X}|}$ possibili ipotesi distinte. Se fissiamo un esempio X e un'ipotesi h , metà di queste ipotesi sarà in disaccordo con h su X , mentre l'altra metà sarà in accordo.

Esempio

Consideriamo $\mathcal{X} = \{X_1, X_2, X_3\}$, ci concentriamo su un esempio randomico (facciamo X_2), e assumiamo $h(X_2) = 0$, le funzioni in disaccordo con h su X_2 sono:

Function	$f(\mathbf{x}_1)$	$f(\mathbf{x}_2)$	$f(\mathbf{x}_3)$
f_1	0	0	0
f_2	0	0	1
f_3	0	1	0
f_4	0	1	1
f_5	1	0	0
f_6	1	0	1
f_7	1	1	0
f_8	1	1	1

In generale, per qualsiasi X ed etichetta di h , metà delle $2^{|\mathcal{X}|}$ funzioni sono in disaccordo con h .

Quindi abbiamo dimostrato che:

$$\sum_f E_{ote}(\mathcal{L}_a | X, f) = \frac{1}{2} 2^{|\mathcal{X}|} \sum_{X \in \mathcal{X}-X} P(X)$$

Osservazione Il teorema assume che si possa mediare su tutte le possibili f ma nel mondo non tutte le ipotesi sono ugualmente importanti. Questo è un risultato interessante, perché dimostra che l'errore medio fuori campione di un algoritmo di apprendimento è **indipendente dall'algoritmo stesso**. Ciò significa che **nessun algoritmo di apprendimento è migliore di un altro** quando si calcola la media su tutti i possibili problemi.

2 Selezione e valutazione di modelli

2.1 Accuracy e Error rate

Il modo più semplice per misurare la bontà di un modello è quella di contare il numero di predizioni sbagliate che esso compie.

$$E = \frac{a}{m}$$

Dove a solo il numero di predizioni sbagliate ed m le predizioni totali E sarà il suo **error rate**.

L'**accuratezza** è calcolata:

$$accuracy = 1 - E = \frac{m - a}{m}$$

L'errore quando viene valutato sul training set è chiamato **empirical error** o **training error**. Quando calcolo l'errore su un data set indipendente si chiamerà **generalization error** o **test error**.

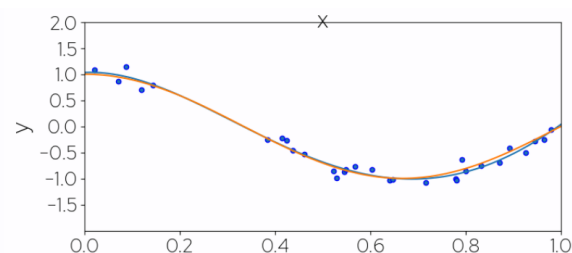
Nota

L'obiettivo principale di un algoritmo di apprendimento è quello di **minimizzare l'errore di generalizzazione**.

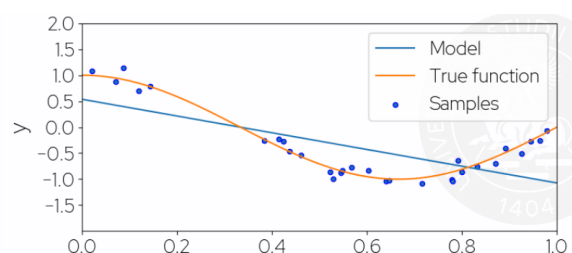
Durante l'addestramento non possiamo calcolare direttamente il **generalization error**, perché non abbiamo accesso ai nuovi esempi, quindi lo approssimiamo minimizzando il **training error**.

2.2 Overfitting

Quando un **modello si adatta troppo bene ai dati di training**, è probabile che alcune peculiarità degli esempi di training vengano assunte come proprietà generali dei dati. Questo porta a un modello che funziona bene sul set di training ma male sui nuovi campioni. Questo fenomeno è chiamato **overfitting**.



Il **fenomeno opposto**, in cui il modello è troppo semplice per catturare i modelli sottostanti nei dati, è chiamato **underfitting**.



L'**underfitting** è solitamente facilmente risolvibile utilizzando un modello più complesso. L'**overfitting** è più difficile da risolvere, poiché richiede un attento equilibrio tra complessità del modello e capacità di generalizzazione. Ciò nonostante, la maggior parte degli algoritmi di apprendimento dispone di meccanismi per prevenire l'overfitting, come **regularization techniques** o **early stopping**.

In genere si preferisce fare un leggero overfitting per avere una capacità espressiva sufficiente.

2.3 Metodi di valutazione

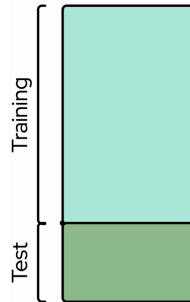
Possiamo valutare le prestazioni di un modello misurandone l'**errore di generalizzazione** su nuovi campioni. Tuttavia, non abbiamo accesso a nuovi campioni durante l'addestramento.

Utilizziamo invece diverse tecniche per stimare l'errore di generalizzazione sulla base dei dati di addestramento.

2.3.1 Validazione Hold-out

Divido l'insieme dei dati in due parti:

- **Training set:** dati che darò in pasto all'algoritmo per l'apprendimento.
- **Test set:** dati che utilizzerò per valutare.

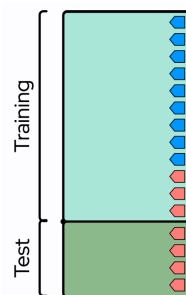


```
1 from sklearn.model_selection import train_test_split
2 ...
3
4 #Assume X,y contains the dataset
5
6 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
    random_state=42)
```

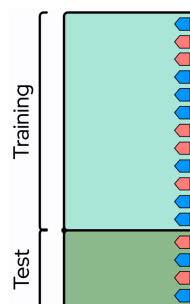
Come effettuare lo split:

- **Simple split**

I primi n dati gli prendo come training e i restanti come test. Ma se ho dei dati non casuali rischio di avere un test set non veritiero.



- **Stratified split** Si mescolano i dati prima di estrarli poi procedo a prendere i primi n come training e il restante come test.

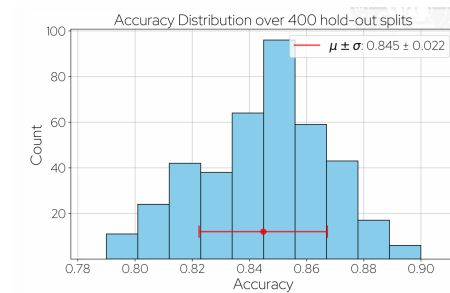


```

1  from model_selection import train_test_split
2  ...
3
4  #Assume X,y contains the dataset
5
6  X_train, X_test, y_train, y_test = train_test_split(X, y, test_size
7  =0.3, shuffle=True, #(default) sufficient for large datasets
8  stratify=y, #useful for small and/or imbalanced datasets
   random_state=42)

```

Se ripetiamo l'esperimento con il metodo di valutazione hold-out molte volte otterremo tanti risultati diversi, perché training e test set cambiano.



Per ottenere una stima dell'errore di generalizzazione più affidabile possiamo ripetere la validazione hold-out molteplici volte e facciamo la media dei risultati.

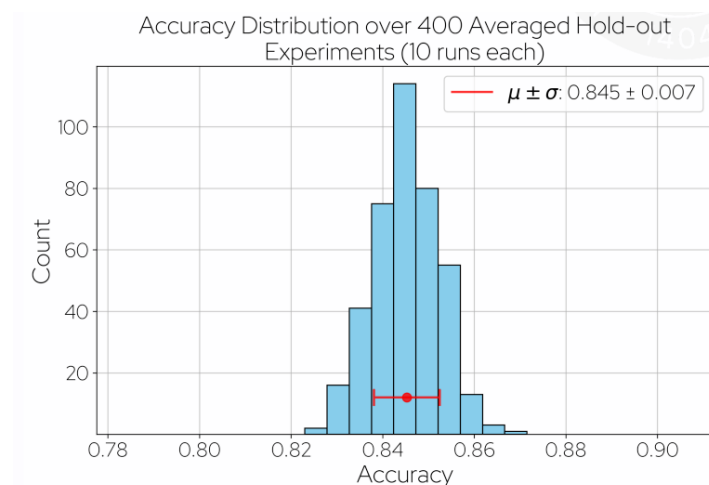
La media di n r.v. **indipendenti**, ognuno con media μ e varianza σ^2 è una nuova r.v. con:

Media

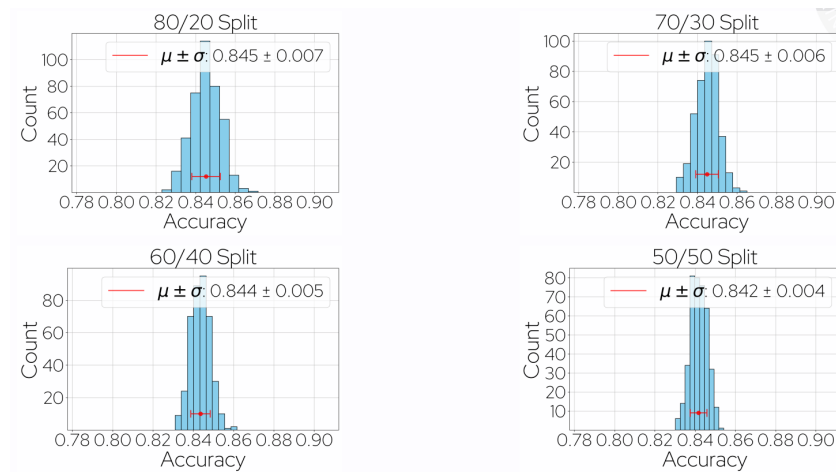
$$\frac{1}{n} \sum_{i=1}^n E_i = \frac{1}{n} \sum_{i=1}^n \mu = \mu$$

Varianza

$$\frac{1}{n^2} \sum_{i=1}^n \sigma^2 = \frac{\sigma^2}{n}$$

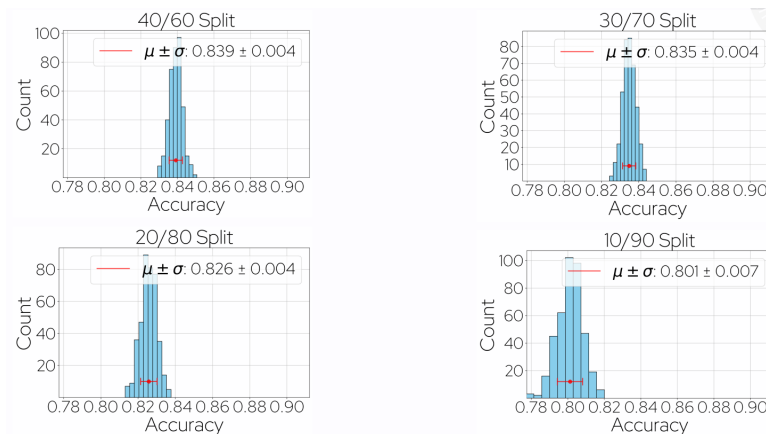


Modifica del rapporto di divisione Una cosa che influisce sull'ampiezza della campana è come stiamo separando il training set dal test set.



Nel momento in cui mettiamo più esempi nel training set otterremo un classificatore che dovrebbe essere più bravo a predire.

All'aumentare della percentuale di esempi nel test set avremo un'accuratezza minore ma la varianza diminuisce.

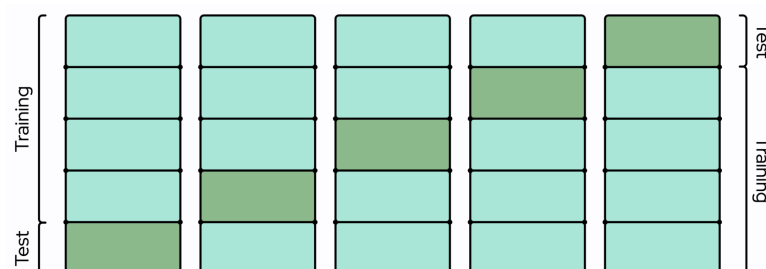


Continuando ad aumentare la percentuale di esempi per il test set la media andrà sempre peggiorando ma la varianza non migliorerà all'infinito anzi ricomincerà ad aumentare.

Se il dataset è grande abbastanza, la **validazione hold-out** è un **semplice** ed **efficace** metodo per stimare l'errore di generalizzazione.

Ma, se il **dataset** è **piccolo**, per tenere da parte una quantità sostanziale di dati per il testing potrebbe portare ad una stima non precisa dell'errore di generalizzazione. In questi casi si può usare la **cross-validation**.

2.3.2 Cross validation



Dividiamo il dataset in k parti, **ripetiamo il processo di hold-out** k volte. La prima volta utilizziamo i primi $k - 1$ **fold** come training set e l'ultimo come test set, la volta successiva utilizziamo il penultimo come test set e tutti gli altri come training set e così via. Alla fine facciamo una **media** e otteniamo una **stima dell'errore di generalizzazione**.

Questa tecnica ha il vantaggio che l'algoritmo di apprendimento ha avuto l'opportunità di vedere, almeno una volta, tutti gli esempi.

Metodo più flessibile

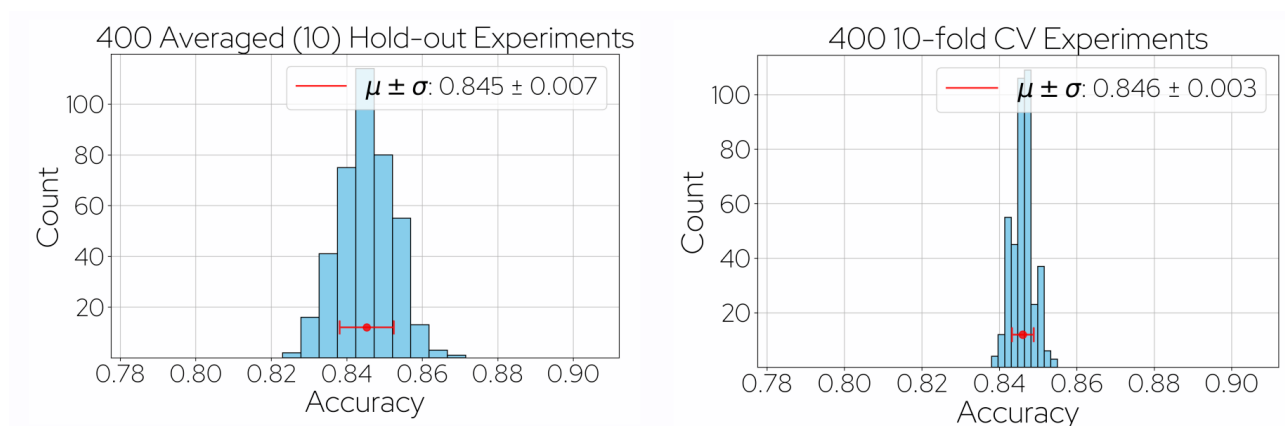
```
1 from sklearn.model_selection import KFold
2 from sklearn.linear_model import LogisticRegression
3 from sklearn.metrics import accuracy_score
4 import numpy as np
5
6 # Assume X,y contains the dataset
7
8 clf = LogisticRegression()
9
10 kf = KFold(n_splits=5, shuffle=True, random_state=42)
11 accuracies = []
12 for train_index, test_index in kf.split(X):
13     X_train, X_test = X[train_index], X[test_index]
14     y_train, y_test = y[train_index], y[test_index]
15
16     clf.fit(X_train, y_train)
17     score = accuracy_score(y_test, clf.predict(X_test))
18     accuracies.append(score)
19
20 print(f"x-val accuracy: {np.array(accuracies).mean()}")
```

Per sapere in modo veloce l'errore di generalizzazione.

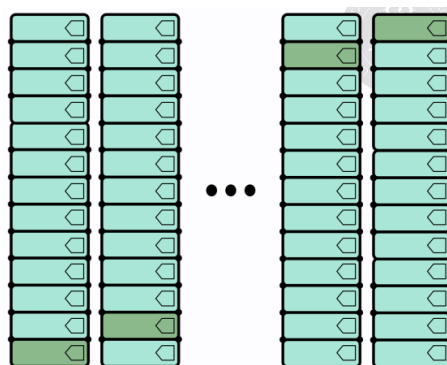
```
1 from sklearn.model_selection import cross_val_score
2 from sklearn.linear_model import LogisticRegression
3
4 # Assume X,y contains the dataset
5
6 clf = LogisticRegression()
7 scores = cross_val_score(clf, X, y, cv=5)
8 print(f"x-val accuracy: {scores.mean()}")
```

Differenza tra hold-out e cross-validation: Nella **cross validation**, ogni campione viene utilizzato per il test **una sola volta**, mentre se **ripetiamo hold-out**, alcuni campioni possono essere utilizzati **più volte per il test** e altri potrebbero **non essere mai utilizzati per il test**. Ciò si traduce in una stima più affidabile dell'errore di generalizzazione (**varianza minore**).

Se il **dataset è sufficientemente grande**, la differenza è trascurabile



2.3.3 Leave-One-Out



Leave-One-Out è un caso estremo di cross-validation dove **ogni campione viene utilizzato per il test una sola volta**, mentre tutti gli altri campioni vengono utilizzati per l'addestramento. Di conseguenza, l'**errore di generalizzazione** viene stimato con una **varianza molto bassa**. Tuttavia, il **costo computazionale** è molto elevato, poiché è necessario addestrare il modello m volte, dove m è il numero di campioni nel set di dati.

2.3.4 Bootstrapping

Dato un set di dati D con campioni m , il nostro obiettivo è stimare l'**errore di generalizzazione** di un modello addestrato sull'**intero set di dati**. Tuttavia, la validazione hold-out e la cross-validation richiedono di addestrare il modello su un sottoinsieme di dati, il che può portare a una stima distorta dell'errore di generalizzazione. Per superare questo problema, possiamo utilizzare il **bootstrapping**. L'idea è quella di **creare più campioni bootstrap** dal set di dati originale D , ciascuno contenente m campioni, campionando con restituzione.

Definizione

Un **campione bootstrap** è un campione ottenuto campionando con sostituzione dal set di dati originale un numero di esempi pari alla dimensione del set di dati originale.

Nota

I **campioni bootstrap non sono disgiunti**. Infatti, ogni campione è ottenuto campionando con sostituzione dal dataset originale, quindi **alcuni campioni potrebbero essere ripetuti** nello stesso campione bootstrap e **altri potrebbero non comparire in esso**.

In realtà è facile calcolare la probabilità che un campione non venga selezionato in un campione bootstrap:

$$P(\text{sample not selected}) = \left(1 - \frac{1}{m}\right)^m$$

Per valori grandi di m , questa probabilità è approssimativamente $e^{-1} \approx 0.3679$, il che significa che in media, circa il **37% dei campioni non viene selezionato** in un campione bootstrap.

Sia D' un campione bootstrap ottenuto da D . Possiamo **addestrare un modello** su D' e **valutarne le prestazioni** sui campioni in $D - D'$ che non sono stati selezionati nel campione bootstrap. In questo modo, possiamo ottenere una stima dell'errore di generalizzazione del modello addestrato su un campione che ha la stessa dimensione del dataset originale.

Nota

Il bootstrapping è **particolarmente utile** quando il **set di dati è piccolo** o quando non esiste un modo efficace per suddividerlo in set di training e test. Quando il set di dati è più grande, il bootstrapping di solito non è necessario e vengono utilizzate la validazione hold-out o la validazione incrociata.

2.4 Tuning dei parametri

Uno dei motivi per cui si utilizzano i metodi di valutazione sopra descritti è la **selezione del modello migliore** tra un insieme di modelli candidati. Questo può significare selezionare il **miglior algoritmo di apprendimento** o selezionare i **migliori iperparametri** per un dato algoritmo di apprendimento.

Definizione

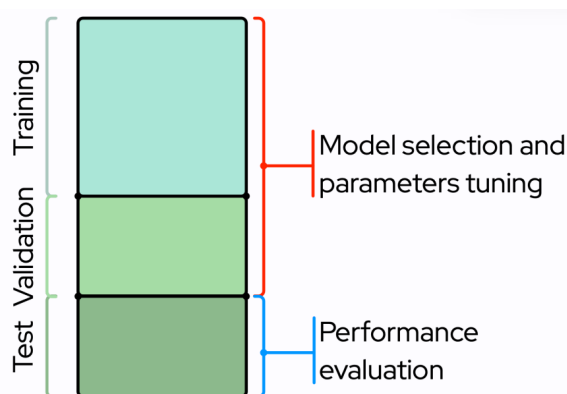
I **parametri** sono gli "oggetti" che l'algoritmo di apprendimento manipola per adattare il modello ai dati, come i pesi in una rete neurale o i coefficienti in un modello di regressione lineare. Gli **iperparametri** sono parametri dell'algoritmo di apprendimento stesso, come l'altezza di un albero decisionale o il parametro di regolarizzazione in un modello di regressione lineare.

La stima dell'errore di generalizzazione utilizzando le tecniche descritte può essere utilizzata per due scopi:

- **Selezione del modello o ottimizzazione dei parametri:** ovvero, selezionare il modello migliore tra un insieme di modelli candidati o selezionare i migliori iperparametri per un dato algoritmo di apprendimento.
- **Stima delle prestazioni:** ovvero stimare le prestazioni del modello finale sulla base di dati non visibili.

Per capire quale è l'algoritmo migliore facciamo un esperimento, prendiamo il dataset, facciamo l'addestramento prima con un set di iperparametri poi con un altro set, valutiamo l'errore di generalizzazione in entrambi i casi e scegliamo quello con l'errore più basso.

Così facendo stiamo valutando basandoci sul test set per fare una scelta quindi bisogna fare **attenzione a non sovraadattare il modello** al set di test. Ciò significa che non dovremmo utilizzare il **set di test per ottimizzare gli iperparametri** del modello, poiché ciò porterebbe a una stima distorta dell'errore di generalizzazione del modello finale.



Per non utilizzare il test set per decidere quali iperparametri utilizzare si divide il dataset in **3 parti**:

- **Training set**
- **Validation set:** una parte di esempi messa da parte per fare la stima degli **iperparametri**
- **Test set:** utilizzato alla fine di tutto quando il modello è stato già scelto per sapere quanto vale l'**errore di generalizzazione** del modello

Esempio

Supponiamo di voler selezionare i migliori iperparametri per un algoritmo di apprendimento automatico. Si dispone di un set di 1000 iperparametri tra cui scegliere e si desidera misurare l'errore utilizzando il set di test. Poiché abbiamo visto che questo potrebbe rappresentare un problema, teniamo da parte anche un set aggiuntivo di esempi, il set di validazione, per misurare l'errore sul classificatore scelto.

Esempio

Poiché sia i set di validazione che quelli di test sono **campioni finiti**, la misurazione dell'errore su di essi introdurrà un po' di rumore nella stima dell'errore di generalizzazione, ovvero:

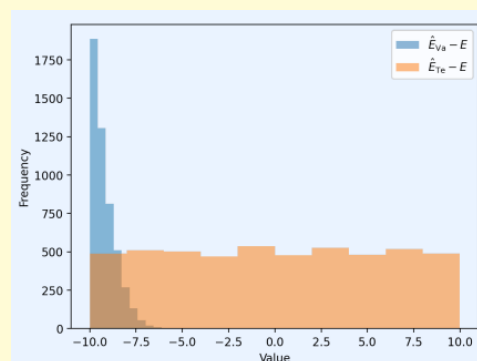
$$\hat{E}_{Tc} = E + \epsilon_{Te}, \quad \hat{E}_{Val} = E + \epsilon_{Val}$$

Si scelgono quindi gli iperparametri che riducono al minimo l'errore sul set di convalida.

Domanda: alla fine del processo, i due errori $\hat{E}_{Te}$ e $\hat{E}_{Val}$ saranno ancora stimatori senza bias dell'errore di generalizzazione? In altri termini, $\mathbb{E}[\hat{E}_{Te}] = \mathbb{E}[\hat{E}_{Val}] = E$?

La media dello stimatore basata sul Validation set avrà un bias, invece quella basata sul Test set sarà senza bias.

```
1 # 1000 hyper-parameters to set, each yielding an error in (0, 100)
2 means = np.random.uniform(0, 100, 1000)
3
4 valid_error_dist = []
5 test_error_dist = []
6
7 N = 5000
8 for i in range(N):
9     samples = np.array(
10         [
11             [
12                 m,
13                 m + np.random.uniform(-10, 10) #E_val
14                 m + np.random.uniform(-10, 10) #E_test
15             ]
16             for m in means
17         ]
18     )
19
20 #selects the best hp according to R_val
21 amin = np.argmin(samples[:, 1])
22
23 valid_error_dist.append(samples[amin, 1] - samples[amin, 0])
24 test_error_dist.append(samples[amin, 2] - samples[amin, 0])
```



2.5 Misurazione delle performance

Diversi indicatori di performance riflettono le diverse esigenze dei compiti e producono risultati di valutazione diversi. La scelta dell'indicatore di performance corretto è fondamentale per una corretta valutazione del modello.

2.5.1 Mean Square Error (MSE)

L'errore quadratico medio è una misura di prestazione comune per le attività di regressione. In generale, se si conosce la **distribuzione dei dati** $\mathcal{D}$, l'errore quadratico medio di una funzione è definito come:

$$E(f; \mathcal{D}) = \mathbb{E}_{(X,y) \sim \mathcal{D}}[(f(X) - y)^2] = \int_{(X,y) \sim \mathcal{D}} (f(X) - y)^2 p(X, y) dX dy$$

Nella maggior parte dei casi, non conosciamo la distribuzione dei dati $\mathcal{D}$, ma disponiamo di un set di dati $D = \{(X_i, y_i)\}_{i=1}^m$ di m campioni estratti da $\mathcal{D}$. In questo caso, possiamo stimare l'errore quadratico medio come:

$$E(f; D) = \frac{1}{m} \sum_{i=1}^m (f(X_i) - y_i)^2$$

2.5.2 Error rate

Analogamente all'errore quadratico medio, possiamo definire il **tasso di errore** e l'**accuratezza** per le attività di classificazione. La forma generale, supponendo di conoscere la distribuzione dei dati, è:

$$E(f; \mathcal{D}) = \mathbb{E}_{(X,y) \sim \mathcal{D}}[\mathbb{I}(f(X) \neq y)] = \int_{(X,y) \sim \mathcal{D}} \mathbb{I}(f(X) \neq y) p(X, y) dX dy$$

Dove $\mathbb{I}$ è la **funzione indicatrice** che restituisce 1 se la condizione è vera e 0 altrimenti.

Se disponiamo di un set di dati $D = \{(X_i, y_i)\}_{i=1}^m$ di m campioni estratti da $\mathcal{D}$, possiamo stimare il tasso di errore come:

$$E(f; D) = \frac{1}{m} \sum_{i=1}^m \mathbb{I}(f(X_i) \neq y_i)$$

2.5.3 Accuracy

L'accuratezza è il **complementare dell'error rate**:

$$acc(f; \mathcal{D}) = 1 - E(f; \mathcal{D}) = 1 - \mathbb{E}_{(X,y) \sim \mathcal{D}}[\mathbb{I}(f(X) \neq y)]$$

o, considerando un dataset $D = \{(X_i, y_i)\}_{i=1}^m$ di m campioni estratti da $\mathcal{D}$:

$$acc(f; D) = 1 - E(f; D) = 1 - \frac{1}{m} \sum_{i=1}^m \mathbb{I}(f(X_i) \neq y_i)$$

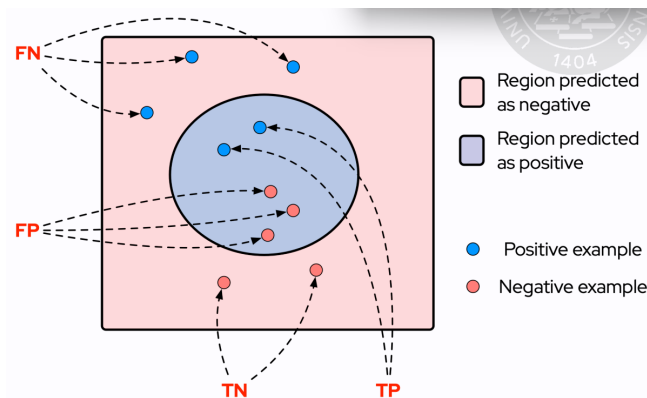
2.5.4 Matrice di confusione

È possibile definire molte altre misure di performance, a seconda del compito e dei requisiti specifici. Per le attività di classificazione, molte di queste misure possono essere derivate dalla **matrice di confusione**.

	Predicted Positive	Predicted Negative
Actual Positive	TP	FN
Actual Negative	FP	TN

con:

- **TP: True Positive** (numero degli esempi che sono stati predetti positivi ed erano veramente positivi)
- **FP: False Positive** (predetti positivi ma in realtà negativi)
- **TN: True Negative** (predetti negativi e veramente negativi)
- **FN: False Negative** (predetti negativi ma in realtà positivi)



2.5.5 Precision, Recall

A volte, gli **errori** non sono **ugualmente importanti**. Ad esempio, in un'attività di diagnosi di cancro, un falso negativo (mancanza di una malattia) può essere più critico di un falso positivo (diagnosi errata di una malattia).

In questo esempio, potremmo voler dare **priorità al richiamo** (recall) (la capacità di identificare tutti i casi di cancro) rispetto alla **precisione** (la capacità di evitare false diagnosi di cancro).

In un'attività di rilevamento dello spam, d'altra parte, potremmo voler dare **priorità alla precisione** (per evitare di etichettare erroneamente email legittime come spam) rispetto al **richiamo** (recall) (per identificare tutti i messaggi di spam).

Più formalmente, la **precisione** è definita come:

$$precision = \frac{TP}{TP + FP} = \frac{TP}{\#predicted\ positives}$$

e il **richiamo** (recall) è definito come:

$$recall = \frac{TP}{TP + FN} = \frac{TP}{\#actual\ positives}$$

Precision e **recall** vengono spesso utilizzati insieme, poiché forniscono informazioni complementari sulle prestazioni del modello. **Di solito sono contraddittorie**, il che significa che l'aumento di una spesso porta a una diminuzione dell'altra.

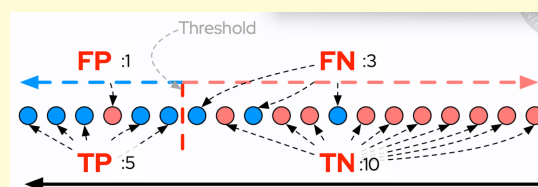
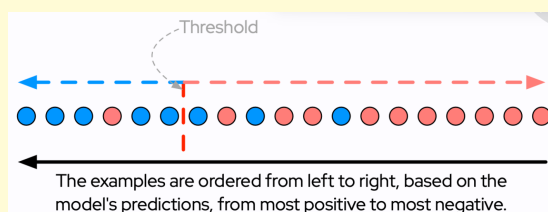
Ad esempio, se iniziamo a classificare **più campioni come positivi**, aumenteremo il numero di veri positivi, ma anche il numero di falsi positivi, **diminuendo** così la **precisione**, ma **aumentando** il **recall**.

D'altra parte, se iniziamo a classificare **meno campioni come positivi**, diminuiranno il numero di falsi positivi, **aumentando** così la **precisione**, ma anche il numero di falsi negativi, **diminuendo** così il **recall**.

2.5.6 P-R Plots

Supponiamo di avere un classificatore che fornisca un **punteggio di confidenza** per ciascun campione, indicando quanto è sicuro che il campione appartenga alla classe positiva. È quindi possibile variare una **soglia** su questo punteggio per ottenere diversi valori di precision e recall.

Esempio

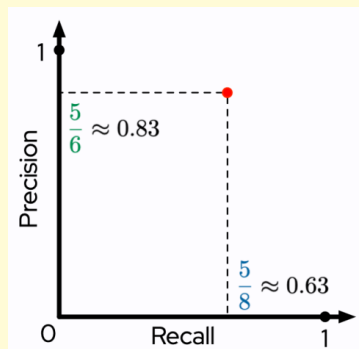


Esempio

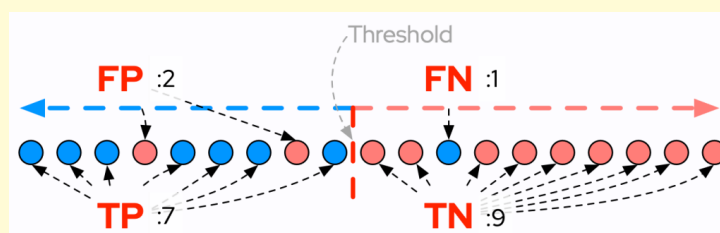
In questo esempio ho $FP = 1$, $FN = 3$, $TP = 5$, $TN = 10$

$$precision = \frac{TN}{TP + FP} = \frac{5}{6}$$

$$recall = \frac{TP}{TP + FN} = \frac{5}{8}$$

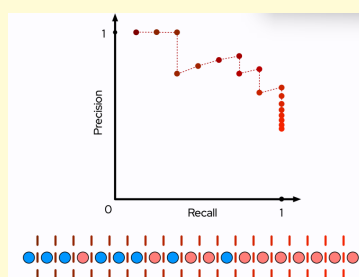
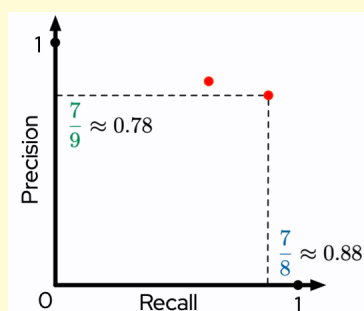


Cambiando la posizione della soglia otterrò:



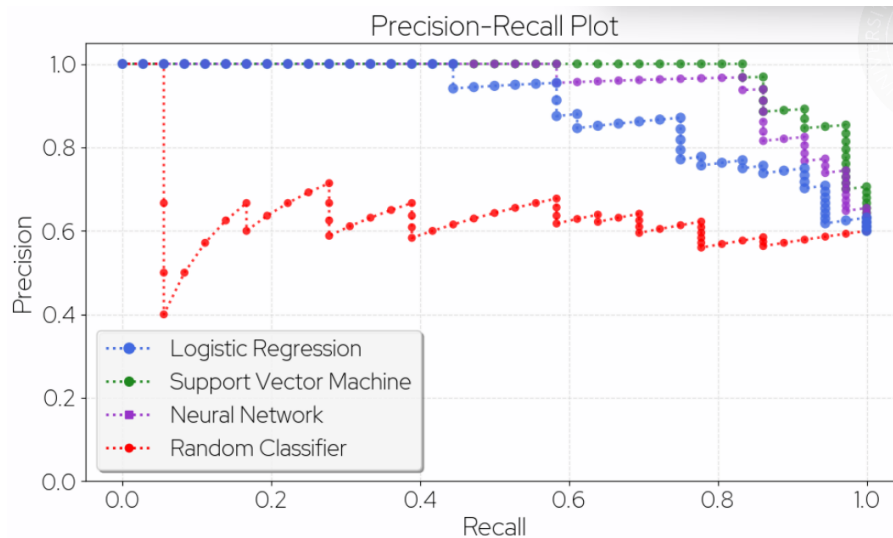
$$precision = \frac{TN}{TP + FP} = \frac{7}{9}$$

$$recall = \frac{TP}{TP + FN} = \frac{7}{8}$$



Dovendo confrontare più classificatori che restituiscono una soglia di confidenza ottengo un grafico

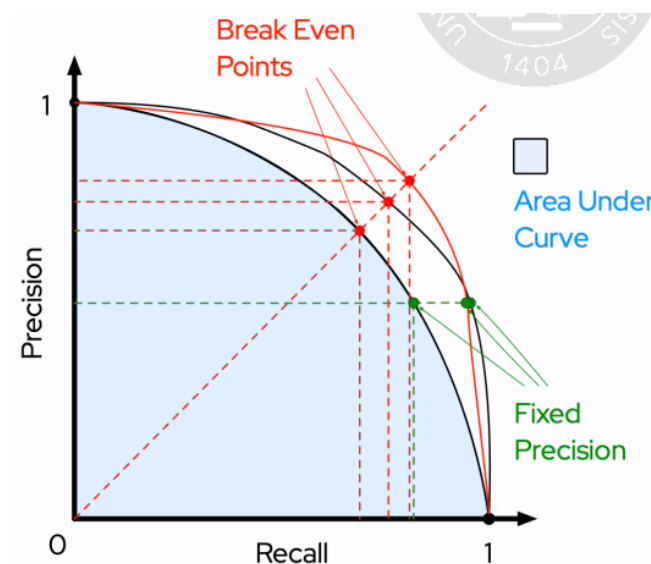
del tipo:



In particolare si può vedere che le **Support Vector Machine** (in verde) dominano tutti gli altri classificatori, se dovessi scegliere un classificatore sceglierei questo perché a parità di threshold vince su tutti gli altri.

Come selezioniamo gli studenti migliori date alcune curve precisione-richiamo?

- **Fissare un dato punto di precision** o recall e selezionare il modello che raggiunge il miglior richiamo o precisione in quel punto.
- Selezionare la curva con l'**area sotto la curva (AUC) più alta** (n.b. difficile da calcolare).
- Selezionare il **miglior punto di break-even**, ovvero il punto in cui precision e recall sono uguali.



F1-score Una misura importante che combina precision e recall è il **punteggio F1**, definito come la **media armonica** di precision e recall:

$$F1 = \frac{2}{\frac{1}{precision} + \frac{1}{recall}} = \frac{2 \times precision \times recall}{precision + recall} = \frac{2 \times TP}{TOT + FP - FN}$$

o in forma più generale (F_β -score):

$$F_\beta = \frac{1 + \beta^2}{\frac{1}{precision} + \frac{\beta^2}{recall}} = \frac{(1 + \beta^2) \times precision \times recall}{\beta^2 \times precision + recall}$$

Media armonica ponderata

La media armonica ponderata di due valori a e b è definita come:

$$\frac{(w_a + w_b)}{\frac{w_a}{a} + \frac{w_b}{b}}$$

Nota

La F-misura definita da F_β misura quanto sono bravo a recuperare gli esempi rispetto ad un utente che pensa che la recall sia β volte più importante della precision.

Supponiamo di voler **calcolare la media** dei contributi di **diverse matrici di confusione** (ad esempio, perché ci troviamo in un problema multi-classe, o vogliamo combinare i risultati di diversi round di convalida incrociata). Indichiamo con P e R le statistiche di **precision** e **recall**.

Questo può essere fatto in due modi:

1. **Macro-averaging**: calcolo la Precision e la Recall per ogni matrice di confusione, denotate come $(P_1, R_1), (P_2, R_2), \dots, (P_n, R_n)$. Prendendo la loro media abbiamo la precision, recall e F_1 macro-averaged:

$$macro-P = \frac{1}{n} \sum_{i=1}^n P_i$$

$$macro-R = \frac{1}{n} \sum_{i=1}^n R_i$$

$$macro-F_1 = \frac{2 \times macro-P \times macro-R}{macro-P + macro-R}$$

2. **Micro-averaging**: calcolare le medie elemento per elemento nelle matrici di confusione, per ottenere $\overline{TP}, \overline{FP}, \overline{TN}, \overline{FN}$, e quindi calcola la micro precision, recall e F_1 come:

$$micro-P = \frac{\overline{TP}}{\overline{TP} + \overline{FP}}$$

$$micro-R = \frac{\overline{TP}}{\overline{TP} + \overline{FN}}$$

$$micro-F_1 = \frac{2 \times micro-P \times micro-R}{micro-P + micro-R}$$

- **Macro-averaging** tratta tutte le matrici di confusione allo stesso modo, indipendentemente dalle loro dimensioni. Questo significa che matrici di confusione di piccole dimensioni possono avere un impatto significativo sul risultato finale.
- **Micro-averaging** attribuisce maggiore peso alle matrici di confusione più grandi, poiché contribuiscono maggiormente al conteggio complessivo di TP, FP, TN e FN. Ciò significa che il risultato finale è più rappresentativo della performance complessiva di tutte le matrici di confusione.

Ciò è importante, ad esempio, nei problemi di classificazione multiclasse in cui le classi possono essere sbilanciate.

Esempio $Macro-F_1$

Assumendo di avere tre matrici di confusione:

$$CM_1 = \begin{bmatrix} 10 & 2 \\ 1 & 7 \end{bmatrix} \quad CM_2 = \begin{bmatrix} 8 & 3 \\ 2 & 6 \end{bmatrix} \quad CM_3 = \begin{bmatrix} 9 & 1 \\ 1 & 8 \end{bmatrix}$$

CM	precision	recall	F1-score
CM ₁	0.9091	0.8333	0.8695
CM ₂	0.8000	0.7273	0.7619
CM ₃	0.9000	0.9000	0.9000
Avg	0.8697	0.8202	0.8438

$$macro-F_1 = \frac{2 \times macro-P \times macro-R}{macro-P + macro-R} = \frac{2 \times 0.8697 \times 0.8202}{0.8697 + 0.8202} \approx 0.8442$$

Esempio $Micro-F_1$

Con le stesse matrici di confusione di prima:

$$CM_1 = \begin{bmatrix} 10 & 2 \\ 1 & 7 \end{bmatrix} \quad CM_2 = \begin{bmatrix} 8 & 3 \\ 2 & 6 \end{bmatrix} \quad CM_3 = \begin{bmatrix} 9 & 1 \\ 1 & 8 \end{bmatrix} \Rightarrow \overline{CM} = \begin{bmatrix} 9 & 2 \\ 1.3333 & 7 \end{bmatrix}$$

La precision e recall micro-averaged sono:

$$micro-P = \frac{9}{9 + 1.3333} = 0.8710 \quad micro-R = \frac{9}{9 + 2} = 0.8182$$

La F_1 micro-averaged è:

$$micro-F_1 = \frac{2 \times micro-P \times micro-R}{micro-P + micro-R} = \frac{2 \times 0.8710 \times 0.8182}{0.8710 + 0.8182} = 0.8438$$

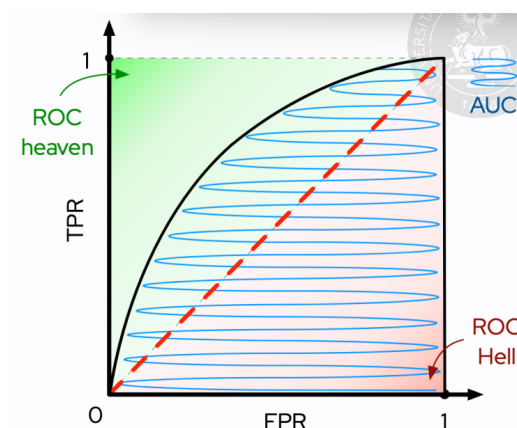
2.5.7 ROC e AUC

Se, invece di utilizzare precision e recall, tracciamo il **false positive rate (FPR)** rispetto al **true positive rate (TPR)**, otteniamo un grafico **ROC** (Receiver Operating Characteristic curve).

$$FPR = \frac{FP}{FP + TN} = \frac{FP}{\#actual\ negative}$$
$$TPR = \frac{TP}{TP + FN} = \frac{TP}{\#actual\ positives}$$

Come nei grafici P-R, diciamo che un learner A è migliore di un learner B se la curva ROC di A racchiude interamente la curva ROC di B . Tuttavia, quando esistono intersezioni, nessun learner è generalmente migliore dell'altro.

Un modo per confrontare le curve ROC intersecate è calcolare le **aree sotto le curve ROC**.



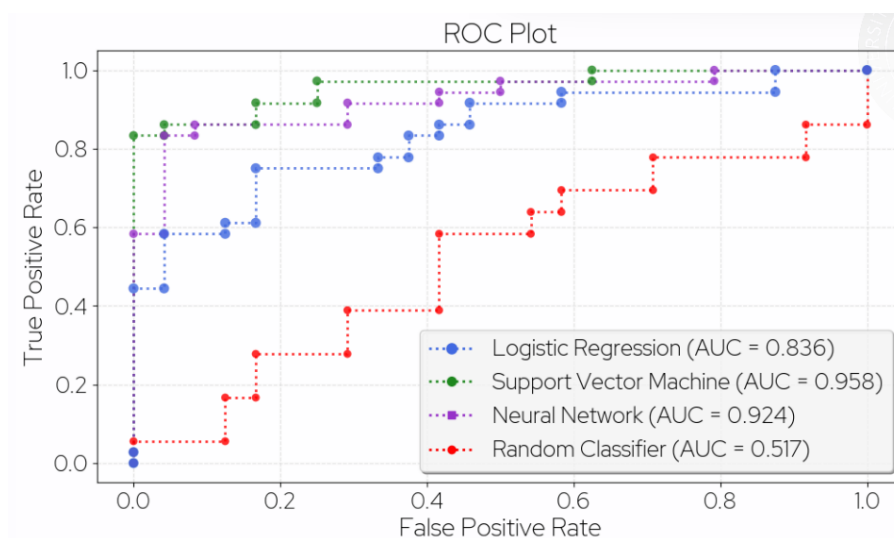
In alto a sinistra (**ROC heaven**) troviamo classificatori che **non sbagliano mai**, in basso a destra (**ROC Hell**) troviamo classificatori che **sbagliano sempre** e al centro sono presenti i peggiori classificatori perché rispondono sempre a caso. Noi **vogliamo stare il più lontano possibile dalla diagonale**.

Area Under the Curve (AUC) Nel caso delle curve ROC, l'area sotto la curva (AUC) è facile da calcolare (anche per le curve teoriche, poiché è sempre non decrescente).

Si può anche dimostrare che è correlata all'errore di ranking del classificatore.

$$1 - AUC = \ell_{rank} \triangleq \frac{1}{m^+ m^-} \sum_{X^+ \in D^+} \sum_{X^- \in D^-} \mathbb{I}[f(X^+) < f(X^-)] + \frac{1}{2} \mathbb{I}[f(X^+) = f(X^-)]$$

Somma su tutti gli esempi positivi, somma su tutti gli esempi negativi calcoliamo un punto di errore se il ranking dell'esempio positivo è minore del ranking dell'esempio negativo, abbiamo anche un altro termine perché possiamo avere dei casi in cui c'è un pareggio in questo caso assegnano mezzo punto di errore. Facciamo la somma di tutto e per normalizzarlo dividiamo tutto per il numero di esempi positivi moltiplicata per il numero di esempi negativi.



```

1 # ...
2 # Assume X_train, X_test, y_train, y_test are defined
3
4 clf = LogisticRegression()
5 clf.fit(X_train, y_train)
6
7 y_scores = clf.predict_proba(X_test)[: , 1]
8
9 # Compute ROC curve and AUC
10 fpr, tpr, _ = roc_curve(y_test, y_scores)
11 roc_auc = auc(fpr, tpr)
12
13 # You can easily plot the ROC curve using matplotlib
14 plt.plot(fpr, tpr)
15
16 # Alternatively, you can directly use roc_auc_score
17 roc_auc2 = roc_auc_score(y_test, y_scores)
18
19 # For Precision-Recall, we don't usually compute
20 # AUC, but we can compute the average precision score
21 avg_precision = average_precision_score(y_test, y_scores)

```

2.6 Cost-Sensitive Error rates

A volte si potrebbe voler adattare l'accuratezza (o l'errore) in modo che siano **sensibili** ai diversi costi di errore per FP e FN . Si consideri $cost_{xy}$ il costo di una classificazione errata x come y .

Per un compito di classificazione binaria potremmo avere la matrice $cost$:

	Predicted as 0	Predicted as 1
Actual 0	0	$cost_{01}$
Actual 1	$cost_{10}$	0

Quindi, supponendo che la classe sia positiva, si può definire il **tasso di errore sensibile ai costi** come:

$$E(f; \mathcal{D}; cost) = \frac{1}{m} \left(\sum_{(X,y) \in \mathcal{D}^+} \mathbb{I}[f(X) \neq y] \times cost_{01} + \sum_{(X,y) \in \mathcal{D}^-} \mathbb{I}[f(X) \neq y] \times cost_{10} \right)$$

Allo stesso modo, possiamo definire il tasso di errore e l'accuratezza basati sulla distribuzione.

2.7 Comparison Test

Confrontare i risultati di diversi learner **non** è semplice per diversi motivi:

- Vogliamo valutare le **prestazioni di generalizzazione**, ma abbiamo accesso solo a un **set di test finito**.
- Le prestazioni del modello possono variare tra **diversi set di test**, anche se hanno le stesse dimensioni.
- La **casualità** intrinseca di molti algoritmi di apprendimento significa che possono produrre modelli diversi se addestrati più volte sugli stessi dati.

L'**Hypothesis testing** consente un rigoroso confronto statistico tra diversi modelli.

2.7.1 Test binomiale

Dato un modello f con un **tasso di errore reale sconosciuto** ϵ , vogliamo verificare se il modello ha prestazioni significativamente migliori rispetto a una determinata soglia di tasso di errore $\epsilon_0 \in [0, 1]$ e ipotizziamo di osservare un **tasso di errore empirico** $\hat{\epsilon}$ su un set di test di dimensioni m tali da consentire un modello f .

Formalizziamo questo come un test di ipotesi:

- **Null hypothesis** (il modello non è migliore di ϵ_0):

$$H_0 : \epsilon \geq \epsilon_0$$

- **Alternative hypothesis** (il modello è migliore di ϵ_0):

$$H_1 : \epsilon < \epsilon_0$$

La difficoltà di tutto questo sta nel fatto che ϵ è sconosciuto, quindi non possiamo calcolare direttamente la probabilità di osservare un tasso di errore empirico $\hat{\epsilon}$ dato un tasso di errore reale ϵ . Poter assumere l'**ipotesi nulla** ci permette di fare dei ragionamenti probabilistici. L'**ipotesi alternativa** è quella che vogliamo dimostrare.

Sia E la variabile casuale che conta il numero di errori fatti dal modello sul test set. Sotto l'ipotesi nulla H_0 , E segue una distribuzione binomiale con parametri m e ϵ_0 : $E \sim \text{Bin}(m, \epsilon_0)$

Nota

Di seguito, indicheremo con $\hat{\epsilon}$ il tasso di errore osservato e con $\hat{e} = \hat{\epsilon} \cdot m$ il numero di errori.

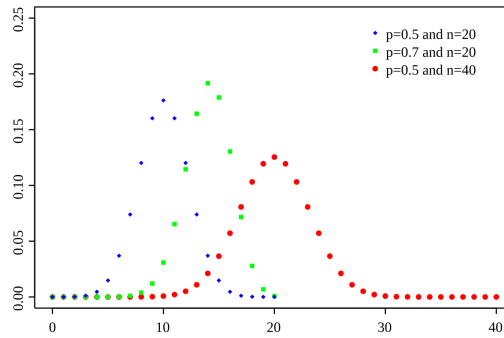


Figura 1: Distribuzione binomiale

Decision Rule

Rifiutare l'ipotesi nulla H_0 a **livello di significatività** α (o con **confidenza** $1 - \alpha$) se:

$$P(E \leq \hat{e} \mid \epsilon = \epsilon_0) < \alpha$$

In altre parole, se è **improbabile** (sotto l'ipotesi nulla H_0) osservare un numero di errori pari o inferiore a $\hat{e}$, allora concludiamo che il modello **funziona meglio** di ϵ_0 .

In caso contrario, **non rifiutiamo** H_0 .

P-value

Il **p-value** di un test statistico è la probabilità di **osservare** una statistica del test **altrettanto estrema o più estrema di quella effettivamente osservata**, supponendo che l'ipotesi nulla sia vera.

Il **p-value**, nel nostro caso, è definito come:

$$p\text{-value} = P(E \leq \hat{e} \mid \epsilon = \epsilon_0) = \sum_{k=0}^{\hat{e}} \binom{m}{k} \epsilon_0^k (1 - \epsilon_0)^{m-k}$$

La probabilità di osservare esattamente k successi su m esperimenti.

Se il p-value è **minore** di α , il risultato è considerato statisticamente significativo e si rifiuta l'ipotesi nulla H_0 .

Esempio

Assumiamo di avere un modello che mostra un tasso di errore del 20%: $\hat{e} = 0.2$ su un set di test di dimensione $m = 100$, cioè compie $\hat{e} \times 100 = 20$ errori sul set di test. Vogliamo verificare l'ipotesi che il vero tasso di errore è significativamente minore di 0.3.

L'ipotesi nulla è:

$$H_0 : \epsilon \geq 0.3$$

L'ipotesi alternativa è:

$$H_1 : \epsilon < 0.3$$

Sotto l'ipotesi nulla, la probabilità di osservare $\hat{e}$ errori è data da:

$$P(\hat{e}; \epsilon_0) = \binom{m}{\hat{e}} \epsilon_0^{\hat{e}} (1 - \epsilon_0)^{m-\hat{e}}$$

Il p-value del test è data da:

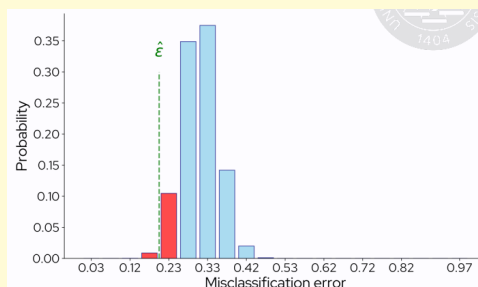
$$P(e \leq \hat{e} \mid \epsilon = 0.3) = \sum_{k=0}^{20} \binom{100}{k} (0.3)^k (1 - 0.3)^{100-k} = 0.0165$$

Esempio

Se impostiamo il livello di significatività a $\alpha = 0.05$, possiamo rifiutare l'ipotesi nulla H_0 fino a quando $0.0165 < 0.05$. Questo significa che abbiamo abbastanza prove per concludere che il modello performa significativamente meglio di un tasso di errore del 30%.

Fissato un **livello di confidenza** α , in rosso nella figura è mostrato la barra corrispondente alla coda sinistra della distribuzione binomiale tale che la probabilità cumulativa è minore o uguale a α .

Poiché $\hat{\epsilon}$ cade nella regione, **possiamo rifiutare l'ipotesi nulla**.



Nel caso in cui volessimo ripetere il test molteplici volte, vorremmo calcolare il tasso di errore massimo $\hat{\epsilon}$ che possiamo osservare per rifiutare ancora l'ipotesi nulla.

Questo valore è chiamato **valore critico** e, per il test binomiale, è definito come il più grande valore di $\hat{\epsilon}$ tale che il p-value è minore o uguale a α :

$$\bar{\epsilon} = \max_{\epsilon} \epsilon \quad \text{subject to} \quad \sum_{i=0}^{\lfloor \epsilon \times m \rfloor} \binom{m}{i} \epsilon^i (1 - \epsilon)^{m-i} \leq \alpha$$

2.7.2 Test di Student

Spesso otteniamo più tassi di errore nei test da cross-validation o da hold-out validation ripetuti. Sia $\hat{\epsilon}_1, \hat{\epsilon}_2, \dots, \hat{\epsilon}_n$ i tassi di errore osservati in k ripetizioni indipendenti del test. Possiamo facilmente calcolare la **media** e la **deviazione standard** dei tassi di errore osservati:

$$\mu = \frac{1}{k} \sum_{i=1}^k \hat{\epsilon}_i$$
$$\sigma^2 = \frac{1}{k-1} \sum_{i=1}^k (\hat{\epsilon}_i - \mu)^2$$

Nota

Per essere rigorosi, l'hold-out ripetuto e (soprattutto) la cross-validation non sono indipendenti, poiché i campioni sono estratti dallo stesso set di dati. Tuttavia, se il set di dati è sufficientemente ampio, la dipendenza è trascurabile e la conclusione rimane valida.

Possiamo considerare questi valori come k campioni indipendenti e identicamente distribuiti (i.i.d.) da una distribuzione normale con media μ e varianza σ^2 e quindi la variabile:

$$\tau = \sqrt{k} \frac{\mu - \epsilon_0}{\sigma}$$

è distribuita come una distribuzione di Student con $k - 1$ gradi di libertà. La distribuzione t di Student è una famiglia di distribuzioni parametrizzata dal valore k che è chiamato **gradi di libertà** (degrees of freedom, df) della distribuzione. Un parametro da fissare per dire quale distribuzione voglio usare.

Il **test di Student** è un test statistico utilizzato per determinare se le medie di due gruppi sono significativamente diverse tra loro. Nel nostro caso, possiamo utilizzarlo per verificare se il tasso di errore medio μ è significativamente diverso da una determinata soglia di tasso di errore ϵ_0 .

L'ipotesi nulla è:

$$H_0 : \mu = \epsilon_0$$

L'ipotesi alternativa è:

$$H_1 : \mu \neq \epsilon_0$$

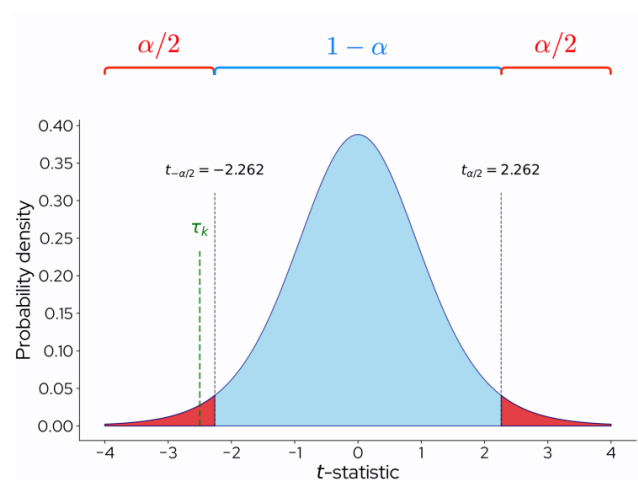
La statistica del test è data da:

$$\tau = \sqrt{k} \frac{\mu - \epsilon_0}{\sigma}$$

Possiamo **rifutare l'ipotesi nulla** a un **livello di significatività** α se τ è **al di fuori** dell'intervallo $[-t_{\alpha/2}, t_{\alpha/2}]$, dove t_z è il valore critico della distribuzione di Student con $k - 1$ gradi di libertà a livello di significatività z . t_z lo ottengo da delle tabelle precompilate e controllo che il mio τ sia fuori dall'intervallo.

Nelle formule, **rifuta l'ipotesi nulla se**:

$$\tau \notin [-t_{\alpha/2}, t_{\alpha/2}]$$



Valori critici comunemente utilizzati

df (k-1)	$\alpha = 0.10$	$\alpha = 0.05$	$\alpha = 0.025$	$\alpha = 0.01$
1	3.078	6.314	12.706	31.821
2	1.886	2.920	4.303	6.965
3	1.638	2.353	3.182	4.541
4	1.533	2.132	2.776	3.747
5	1.476	2.015	2.571	3.365
6	1.440	1.943	2.447	3.143
7	1.415	1.895	2.365	2.998
8	1.397	1.860	2.306	2.896
9	1.383	1.833	2.262	2.821
10	1.372	1.812	2.228	2.764
11	1.363	1.796	2.201	2.718
12	1.356	1.782	2.179	2.681
13	1.350	1.771	2.160	2.650
14	1.345	1.761	2.145	2.624
15	1.341	1.753	2.131	2.602
16	1.337	1.746	2.120	2.583
17	1.333	1.740	2.110	2.567
18	1.330	1.734	2.101	2.552
19	1.328	1.729	2.093	2.539
20	1.325	1.725	2.086	2.528
25	1.316	1.708	2.060	2.485
30	1.310	1.697	2.042	2.457
∞	1.282	1.645	1.960	2.326

Il libro riporta i valori critici usando k come numero di campioni (qui sono riportati i gradi di libertà), e α è già diviso per 2. Quindi, la voce per $k = 5$ e $\alpha = 0.05$ sul libro corrisponde alla voce per $df = 4$ e $\alpha = 0.025$ nella tabella.

```

1 from scipy.stats import t
2 import numpy as np
3
4 # Assume we have k error rates in a list
5 error_rates = [0.2, 0.18, 0.22, 0.19, 0.21]
6 k = len(error_rates)
7 mu = np.mean(error_rates)
8 sigma = np.std(error_rates, ddof=1) # ddof=1 for sample standard deviation
9
10 alpha = 0.05
11
12 # ppf == percent point function, i.e., the inverse of the cumulative
13   distribution function (CDF)
14 t_crit_0 = t.ppf(alpha/2, k-1) # negative crit for left tail
15 t_crit_1 = t.ppf(1 - alpha/2, k-1) # positive critical value for right tail
16
17 tau_k = no.sqrt(k) * (mu - 0.3) / sigma
18
19 if(tau_k < t_crit_0 or tau_k > t_crit_1):
20     print("Reject null hypothesis H0")
21 else:
22     print("Do not reject null hypothesis H0")
23
24 # Alternatively, you can use the ttest_1samp function from scipy.stats
25 from scipy.stats import ttest_1samp
26 t_stat, p_value = ttest_1samp(error_rates, 0.3)
27 if p_value < alpha:
28     print("Reject null hypothesis H0")
29 else:
30     print("Do not reject null hypothesis H0")

```

2.7.3 Cross-Validated Student's t-test

Assumiamo di avere due algoritmi di apprendimento A e B , e denotiamo con $\epsilon_1^A, \dots, \epsilon_k^A$ e $\epsilon_1^B, \dots, \epsilon_k^B$ i tassi di errore ottenuti da **k-fold cross validation**.

Se vogliamo confrontare i due algoritmi, possiamo usare il **k-fold cross-validation Student's t-test**.

L'idea è che, se le performance dei due algoritmi sono le stesse, allora i tassi di errore nei test dovrebbero essere simili nelle stesse suddivisioni training/test: $\epsilon_i^A \approx \epsilon_i^B$ per tutte le $i = 1, \dots, k$.

Il test richiede di calcolare $\Delta_i = \epsilon_i^A - \epsilon_i^B$ per $i = 1, \dots, k$, e quindi calcolare la media μ e la deviazione standard σ della differenza Δ_i .

Note

Nota 1: si presume che i tassi di errore siano indipendenti, il che non è vero nella pratica, ma l'ipotesi è ragionevole se il set di dati è sufficientemente ampio.

Nota 2: la tecnica può essere utilizzata anche con validation hold-out accoppiate (vale a dire, le stesse suddivisioni di formazione/test vengono utilizzate per entrambi gli algoritmi).

Ipotesi nulla: i due algoritmi hanno le stesse prestazioni, cioè, $\mu = 0$.

Ipotesi alternativa: i due algoritmi hanno prestazioni diverse, cioè, $\mu \neq 0$.

La statistica del test è data da:

$$\tau = \frac{\sqrt{k} \cdot \mu}{\sigma}$$

Regola di decisione: rifiutare l'ipotesi nulla H_0 a un livello di significatività α se:

$$\tau \notin [-t_{\alpha/2}, t_{\alpha/2}]$$

dove $t_{\frac{\alpha}{2}}$ è il valore critico della distribuzione di Student con $k-1$ gradi di libertà a livello di significatività $\frac{\alpha}{2}$.

2.7.4 5x2 Cross-Validated t-test

Come notato, l'ipotesi di indipendenza è violata nel caso di CV k-fold. Per risolvere questo problema, è possibile utilizzare metodi più robusti come il **5x2 cross-validated t-test**.

La procedura è la seguente:

1. Eseguire 2-fold cross-validation. Siano $\Delta_{1,1}$ e $\Delta_{1,2}$ le differenze di errore tra i due algoritmi nei due fold.
2. Ripetere questo processo 4 altre volte, ogni volta con una diversa suddivisione dei dati, ottenendo così un totale di 5 paia di differenze in totale. Denotiamo le differenze di errore come $\Delta_{i,1}$ e $\Delta_{i,2}$ per $i = 1, \dots, 5$.
3. Per ognuna delle 5 ripetizioni, calcoliamo la varianza delle due differenze come:

$$\sigma_i^2 = (\Delta_{i,1} - \bar{\Delta}_i)^2 + (\Delta_{i,2} - \bar{\Delta}_i)^2$$

$$\bar{\Delta}_i = \frac{\Delta_{i,1} + \Delta_{i,2}}{2}$$

4. Il test statistico è calcolato usando la differenza tra i due fold della prima ripetizione e la media delle varianze calcolate nelle 5 ripetizioni:

$$\tau = \frac{\Delta_{1,1}}{\sqrt{\frac{1}{5} \sum_{i=1}^5 \sigma_i^2}}$$

5. Sotto l'ipotesi nulla, questa statistica è distribuita secondo una **distribuzione di Student con 5 gradi di libertà**. La regola di decisione è la stessa di prima, ma usando il valore critico $t_{\alpha/2}$ con 5 gradi di libertà.

Nota

Il libro di testo suggerisce di usare la media $0.5(\Delta_{i,1} + \Delta_{i,2})$ al posto di $\Delta_{i,1}$ nella formula per τ andando in contrasto con *original paper by Dietterich*.

```
1 from scipy.stats import t
2
3 # ...
4
5 # Assume errors_A and errors_B are lists of error rates for learners A and B
  # obtained from k-fold cross-validation
6
7 k = len(errors_A)
8 deltas = np.array(errors_A) - np.array(errors_B)
9 mu = np.mean(deltas)
10 sigma = np.std(deltas, ddof=1) # Sample standard deviation
11
12 tau = np.sqrt(k) * mu / sigma
13 t_crit = t.ppf(1 - alpha/2, k-1) # critical value for two-tailed test
14
15 # Decision rule
16 if abs(tau) > t_crit:
17     print("Reject null hypothesis H0: learners A and B have different
      performance")
18 else:
19     print("Do not reject null hypothesis H0: learners A and B have similar
      performance")
```

2.7.5 McNemar's Test

Assumiamo di avere due classificatori, A e B , e vogliamo confrontare le loro prestazioni usando il metodo hold-out. Addestriamo entrambi i classificatori sullo stesso set di addestramento e li valutiamo sullo stesso set di test, così facendo possiamo ottenere la seguente tabella di contingenza:

	Classifier A Correct	Classifier A Incorrect
Classifier B Correct	e_{11}	e_{10}
Classifier B Incorrect	e_{01}	e_{00}

L'idea è che, sotto l'**ipotesi nulla** che i due classificatori abbiano le stesse prestazioni, ci aspettiamo che il **numero di errori fatti da entrambi i classificatori sia simile**, cioè:

$$e_{01} \approx e_{10}$$

Il McNemar's test considera la variabile:

$$\tau_{\chi^2} = \frac{(|e_{10} - e_{01}| - 1)^2}{e_{10} + e_{01}}$$

che segue una distribuzione χ^2 con 1 grado di libertà sotto l'ipotesi nulla.

Regola di decisione: rifiutare l'ipotesi nulla H_0 a un livello di significatività α se:

$$\tau_{\chi^2} > \chi_{\alpha,1}^2$$

dove $\chi_{\alpha,1}^2$ è il valore critico della distribuzione χ^2 con 1 grado di libertà a livello di significatività α .

Tipicamente i valori critici più comuni sono:

$\alpha = 0.10$	$\alpha = 0.05$	$\alpha = 0.01$
2.706	3.841	6.635

2.7.6 Friedman Test

Entrambi i test di Student e McNemar sono progettati per confrontare due algoritmi di apprendimento su un singolo dataset. Tuttavia, in pratica, spesso vogliamo **confrontare più di due algoritmi su più dataset**.

In questo caso, possiamo confrontare ciascuna coppia di algoritmi utilizzando il test di Student o il test di McNemar, ma questo può essere **computazionalmente costoso** e può comportare un aumento del rischio di errori di tipo I (falsi positivi).

Il **test di Friedman** è un test di ranking che consente di confrontare più algoritmi su più set di dati utilizzando un singolo test. Se l'ipotesi nulla viene rifiutata, è necessario utilizzare un **test post-hoc** (come il test di Nemenyi) per identificare le coppie specifiche di algoritmi che presentano prestazioni **significativamente diverse**.

Assumendo di comparare gli algoritmi A , B , e C sui dataset D_1 , D_2 , D_3 e D_4 . Iniziamo usando la **validazione hold-out o la cross-validation per ottenere i tassi di errore** per ogni algoritmo su ogni dataset, poi **ordiniamo gli algoritmi** per ogni dataset in base al loro tasso di errore (il più basso ottiene il rango 1, il secondo più basso ottiene il rango 2, e così via), per ogni set di dati in cui agli algoritmi con lo stesso tasso di errore viene assegnato il rango medio.

Esempio

Supponiamo di avere i seguenti tassi di errore per i tre algoritmi sui quattro set di dati:

Error rates:

Dataset	Algorithm A	Algorithm B	Algorithm C
D_1	0.2	0.3	0.4
D_2	0.1	0.2	0.2
D_3	0.05	0.1	0.2
D_4	0.3	0.4	0.5

Ranks:

Dataset	Algorithm A	Algorithm B	Algorithm C
D_1	1	2	3
D_2	1	2.5	2.5
D_3	1	2	3
D_4	1	2	3
Averages	1	2.125	2.875

Sia k il **numero di algoritmi**, N il numero di dataset, e r_i il rango medio dell'algoritmo i su tutti i dataset. Per calcolare la statistica del test di Friedman, partiamo a calcolare la formula:

$$\tau_{\chi^2} = \frac{12N}{k(k+1)} \left(\sum_{i=1}^k r_i^2 - \frac{k(k+1)^2}{4} \right)$$

Il test statistico segue una distribuzione χ^2 con $k - 1$ gradi di libertà sotto l'ipotesi nulla che tutti gli algoritmi abbiano le stesse prestazioni.

Tuttavia, questa statistica **può essere utilizzata solo quando è di k grandi dimensioni**. Normalmente, la statistica viene corretta utilizzando la **statistica corretta di Iman e Davenport**:

$$\tau_F = \frac{(N-1) \cdot \tau_{\chi^2}}{N(k-1) - \tau_{\chi^2}}$$

Questa statistica corretta segue una distribuzione F con $k - 1$ e $(k - 1)(N - 1)$ gradi di libertà sotto l'ipotesi nulla.

Critical values for the F-test at $\alpha = 0.05$

N	$k = 2$	$k = 3$	$k = 4$	$k = 5$	$k = 6$	$k = 7$	$k = 8$	$k = 9$	$k = 10$
4	10.128	5.143	3.863	3.259	2.901	2.661	2.488	2.355	2.250
5	7.709	4.459	3.490	3.007	2.711	2.508	2.359	2.244	2.153
8	5.591	3.739	3.072	2.714	2.485	2.324	2.203	2.109	2.032
10	5.117	3.555	2.960	2.634	2.422	2.272	2.159	2.070	1.998
15	4.600	3.340	2.827	2.537	2.346	2.209	2.104	2.022	1.955
20	4.381	3.245	2.766	2.492	2.310	2.179	2.079	2.000	1.935

Critical values for the F-test at $\alpha = 0.1$

N	$k = 2$	$k = 3$	$k = 4$	$k = 5$	$k = 6$	$k = 7$	$k = 8$	$k = 9$	$k = 10$
4	5.538	3.463	2.813	2.480	2.273	2.130	2.023	1.940	1.874
5	4.545	3.113	2.606	2.333	2.158	2.035	1.943	1.870	1.811
8	3.589	2.726	2.365	2.157	2.019	1.919	1.843	1.782	1.733
10	3.360	2.624	2.299	2.108	1.980	1.886	1.814	1.757	1.710
15	3.102	2.503	2.219	2.048	1.931	1.845	1.779	1.726	1.682
20	2.990	2.448	2.182	2.020	1.909	1.826	1.762	1.711	1.668

2.7.7 Nemenyi Post-hoc Test

Se l'ipotesi nulla del test di Friedman viene rifiutata, possiamo procedere con un test post-hoc per identificare quali coppie di algoritmi hanno prestazioni significativamente diverse. Il **test di Nemenyi** è una scelta comune a questo scopo.

Il test di Nemenyi confronta le prestazioni di due classificatori osservando la differenza nei loro ranghi medi. Le prestazioni di due classificatori sono considerate significativamente diverse **se i ranghi medi corrispondenti differiscono almeno della differenza critica (CD)**, che si calcola come:

$$CD = q_\alpha \sqrt{\frac{k(k+1)}{6N}}$$

dove k è il numero di algoritmi, N il numero di dataset, e q_α è il valore critico della distribuzione di Studentizzata al livello di significatività α .

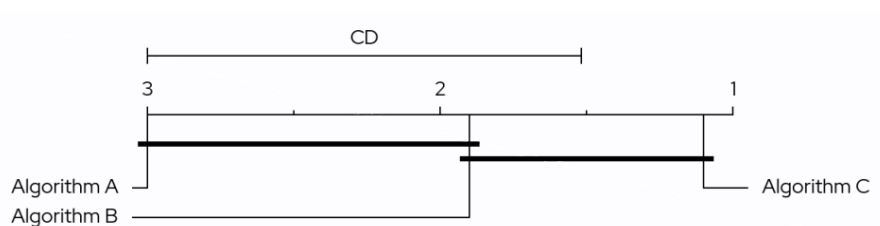
Si ritiene che due algoritmi abbiano prestazioni significativamente diverse se la differenza assoluta dei loro ranghi medi è maggiore del CD:

$$|r_i - r_j| > CD$$

2.7.8 Critical difference diagram

La critical difference viene spesso visualizzata utilizzando un **diagramma della differenza critica**, che mostra con CD la distanza della critical difference tra i ranghi medi degli algoritmi confrontati.

A e B hanno una barra che li collega, quindi non sono significativamente diversi, invece A e C non hanno una barra che li collega, quindi sono significativamente diversi.



2.8 Bias Variance decomposition

La **decomposizione Bias-Varianza** è un modo per comprendere le fonti di errore in un modello di apprendimento automatico.

Per diversi set di apprendimento, il risultato è spesso diverso, sebbene i campioni siano estratti dalla stessa distribuzione. La decomposizione Bias-Varianza ci aiuta a capire perché questo accade scomponendo l'errore in tre componenti:

1. **Bias**: l'errore dovuto a ipotesi errate nel modello di apprendimento, che può portare a dell'underfitting.
2. **Variance**: l'errore dovuto alla sensibilità del modello a piccole fluttuazioni nel set di addestramento, che può portare a un overfitting.
3. **Irreducible error**: l'errore che non può essere ridotto da nessun modello, spesso dovuto al rumore nei dati.

Consideriamo un problema di **regressione** e sia X un **campione di prova**, y_D è l'etichetta associata al dataset D^1 , y l'etichetta della ground truth per X , e $f(X; D)$ la precisione del modello f addestrato su D .

Expected prediction: la previsione prevista per il modello su X è data da:

$$\bar{f} = \mathbb{E}_D[f(X; D)]$$

Varianza:

$$var = \mathbb{E}_D[(f(X; D) - \bar{f})^2]$$

Rumore:

$$\epsilon^2 = \mathbb{E}_D[(y_D - y)^2]$$

Bias: cioè, la differenza tra la previsione prevista e l'etichetta reale:

$$bias^2 = (\bar{f} - y)^2$$

$\mathbb{E}_D$ è la media su tutti i possibili set di addestramento D .

Dimostrazione

Per facilitare la discussione, supponiamo che l'aspettativa di rumore sia zero, ovvero, $\mathbb{E}_D[y_D - y] = 0$.

L'MSE previsto del modello può essere scritto come:

$$\begin{aligned} E(f; D) &= \mathbb{E}_D[(f(\mathbf{x}; D) - y_D)^2] \\ &= \mathbb{E}_D[(\overbrace{f(\mathbf{x}; D) - \bar{f}}^a + \overbrace{\bar{f} - y_D}^b)^2] = \mathbb{E}_D[(a + b)^2] \\ &= \mathbb{E}_D[(f(\mathbf{x}; D) - \bar{f})^2] + \mathbb{E}_D[(\bar{f} - y_D)^2] + \cancel{2\mathbb{E}_D[(f(\mathbf{x}; D) - \bar{f})(\bar{f} - y_D)]} \quad \text{since } y_D \perp f_D \\ &= \mathbb{E}_D[(f(\mathbf{x}; D) - \bar{f})^2] + \mathbb{E}_D[(\bar{f} - y_D)^2] \\ &= var + \mathbb{E}_D[(\bar{f} - y + y - y_D)^2] \\ &= var + \cancel{\mathbb{E}_D[(\bar{f} - y)^2]} + \mathbb{E}_D[(y - y_D)^2] + \cancel{2\mathbb{E}_D[(\bar{f} - y)(y - y_D)]} \quad \text{since } \mathbb{E}_D[y_D - y] = 0 \Rightarrow \mathbb{E}_D[y_D] = y \\ &= var + bias^2 + \epsilon^2 \end{aligned}$$

Calcoliamo l'errore f su tutto il dataset D : vado a prendere la media su tutti i possibili dataset D , della predizione di f meno l'etichetta reale al quadrato (sto calcolando l'errore empirico di f sul test set, sull'esempio specifico X).

Sottraggo e aggiungo $\bar{f}$ all'interno della parentesi quadra e sviluppo il quadrato della somma.

Il terzo termine va a 0 nel momento in cui y_D è indipendente da f_D , vero perché l'etichetta dell'esempio nel dataset è indipendente dalla funzione f perché addestrata sulla parte di training del dataset D , quindi non ha mai visto quell'etichetta rendendola indipendente. (Svolgendo la moltiplicazione ottengo: $\mathbb{E}[f_d \bar{f}] - \mathbb{E}[f_D y_d] - \mathbb{E}[\bar{f}^2] + \mathbb{E}[\bar{f} y_D] \rightarrow \bar{f}^2 - \mathbb{E}[f_D y_D] - \bar{f}^2 + \bar{f} \mathbb{E}_D[y_D] \rightarrow -\mathbb{E}[f_d] \mathbb{E}_D[y_D] + \bar{f} \mathbb{E}_D[y_D] = 0$).

Aggiungo e sottraggo nuovamente il valore vero dell'etichetta e ottengo la media su tutti i dataset.

Bias misura la differenza tra la previsione attesa dell'algoritmo di apprendimento e l'etichetta di ground-truth, che riflette la **capacità di adattamento dell'algoritmo di apprendimento**.

Varianza misura quanto le prestazioni di apprendimento cambiano a causa delle variazioni nel set di addestramento, riflettendo l'**impatto delle fluttuazioni dei dati sul risultato dell'apprendimento**.

Noise rappresenta il limite inferiore dell'errore di generalizzazione atteso che può essere raggiunto da **qualsiasi algoritmo di apprendimento** per il compito dato.

Un altro modo di vedere la cosa è che la decomposizione bias-varianza ci dice che la **performance della generalizzazione** è determinata congiuntamente da:

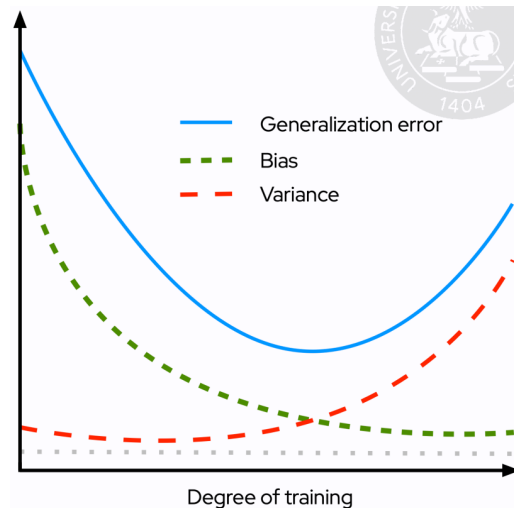
- la capacità dell'algoritmo di apprendimento
- sufficienza dei dati

- e l'errore intrinseco del problema di apprendimento

Per ottenere eccellenti prestazioni di generalizzazione, un modello necessita di un **piccolo bias** (per adattarsi adeguatamente ai dati) e di una **piccola varianza** (per ridurre al minimo l'impatto delle fluttuazioni dei dati).

2.8.1 Bias-Variance Dilemma

Il **dilemma bias-varianza** si riferisce al compromesso tra bias e varianza in un algoritmo di apprendimento: in generale gli algoritmi con un basso bias tendono ad avere un'alta varianza, e viceversa.



3 Linear Models

Sia $X = (x_1, x_2, \dots, x_d)^T$ un'esempio descritto da d variabili.

Un **modello lineare** mira a imparare una funzione che fa predizioni da una **combinazione lineare** delle variabili di input, che è:

$$f(X) = w_1x_1 + w_2x_2 + \dots + w_dx_d + w_{d+1} = \mathbf{w}^T \mathbf{X} + w_{d+1}$$

dove $\mathbf{w} = (w_1, w_2, \dots, w_d)^T$ è il vettore dei pesi e w_{d+1} è il bias.

Il **modello è determinato** una volta che $\mathbf{w}$ e w_{d+1} sono stati **appresi dai dati**.

3.1 Regressione lineare

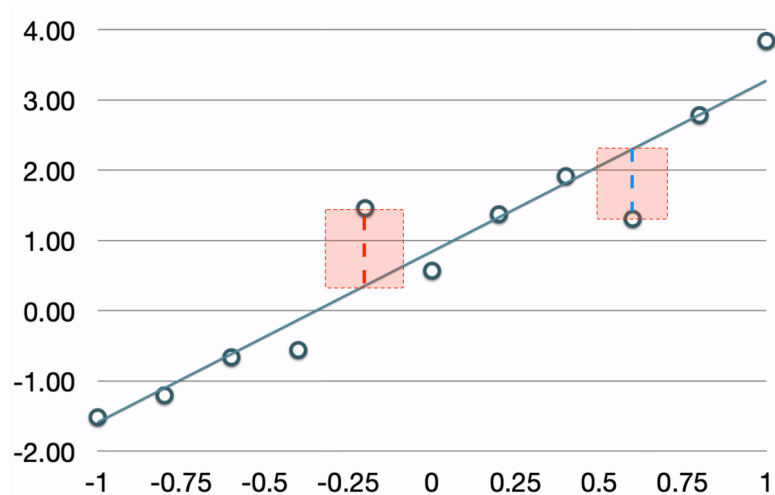
Dato un dataset $D = \{(X_1, y_1), (X_2, y_2), \dots, (X_n, y_n)\}$, dove $X_i = (x_{i1}, x_{i2}, \dots, x_{id})$ e $y_i \in \mathbb{R}$.

La **regressione lineare** mira ad apprendere un **modello lineare** in grado di prevedere con precisione il valore reale delle etichette di output.

L'idea chiave è misurare e minimizzare la differenza tra i valori previsti e i valori effettivi nel dataset:

$$MSE(\mathbf{w}, w_{d+1}) = \frac{1}{n} \sum_{i=1}^n (y_i - f(X_i))^2 = \frac{1}{n} \sum_{i=1}^n (y_i - (\mathbf{w}^T X_i + w_{d+1}))^2$$

3.2 Mean Squared Error and Best Fit Line



x	y
-1	-1.52
-0.8	-1.21
-0.6	-0.67
-0.4	-0.56
-0.2	1.46
0	0.57
0.2	1.37
0.4	1.91
0.6	1.31
0.8	2.78
1	3.84

Nel caso univariato nell'esempio, vogliamo trovare i parametri w_1 e w_2 del modello lineare $f(x) = w_1x + w_2$ che meglio si adatta ai dati.

Idealmente l'equazione lineare varrebbe per ogni riga della tabella a sinistra. Cioè, il nostro obiettivo è trovare w_1 e w_2 tali che:

$$\begin{cases} w_1 \cdot (-1) + w_2 = -1.52 \\ w_1 \cdot (-0.8) + w_2 = -1.21 \\ w_1 \cdot (-0.6) + w_2 = -0.67 \\ \dots \end{cases}$$

3.2.1 Rappresentazione matriciale

Il sistema:

$$\begin{cases} w_1 \cdot (-1) + w_2 = -1.52 \\ w_1 \cdot (-0.8) + w_2 = -1.21 \\ w_1 \cdot (-0.6) + w_2 = -0.67 \\ \dots \end{cases}$$

può essere scritto in forma matriciale come:

$$\begin{bmatrix} x \\ -1 \\ -0.8 \\ -0.6 \\ \vdots \end{bmatrix} \begin{bmatrix} w \\ w_1 \end{bmatrix} + w_2 = \begin{bmatrix} y \\ -1.52 \\ -1.21 \\ -0.67 \\ \vdots \end{bmatrix} = \mathbf{X}\mathbf{w} + w_2$$

3.2.2 Coordinate omogenee

Incorporeremo il bias b nel vettore dei pesi, che diventa $\mathbf{w}' = (w_1, w_2, \dots, w_d, w_{d+1})^T$, e aggiungiamo una colonna di uno alla matrice di input $\mathbf{X}$. Questa rappresentazione è chiamata **homogeneous coordinates**:

$$\begin{bmatrix} x \\ -1 \\ -0.8 \\ -0.6 \\ \dots \end{bmatrix} \begin{bmatrix} w \\ w_1 \\ w_2 \end{bmatrix} = \begin{bmatrix} y \\ -1.52 \\ -1.21 \\ -0.67 \\ \dots \end{bmatrix} \equiv \mathbf{X}\mathbf{w} = y$$

Domanda

Dato l'esempio:

$$\begin{aligned}x_1 &= (1, 2), & y_1 &= 0.8 \\x_2 &= (3, 4), & y_2 &= 0.4 \\x_3 &= (1, 4), & y_3 &= 0.7 \\x_4 &= (2, 2), & y_4 &= 0.9\end{aligned}$$

Quali sono le matrici $\mathbf{X}$ e il vettore $\mathbf{y}$?

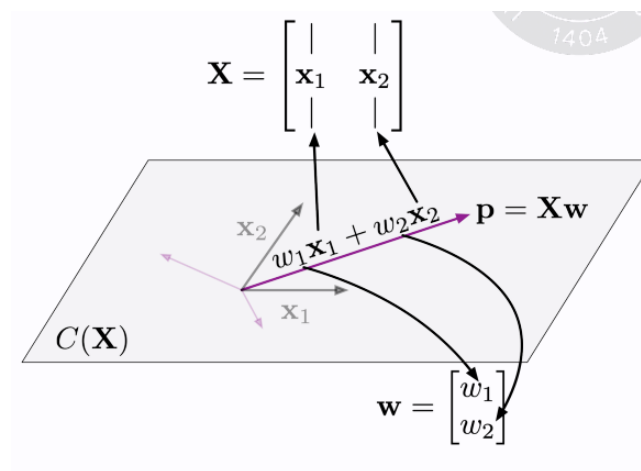
3.2.3 Risolvere il sistema

Nel caso ideale $\mathbf{X}$ è al quadrato e di rango pieno, e l'equazione può essere risolta semplicemente moltiplicando entrambi i membri per l'inverso di $\mathbf{X}$:

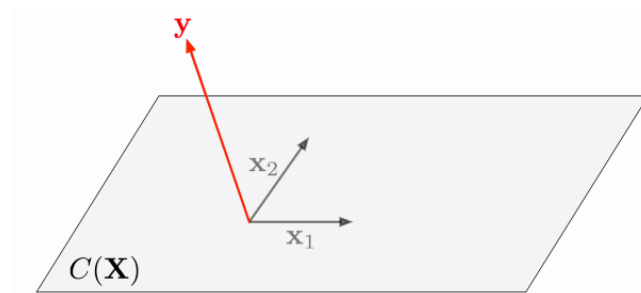
$$\begin{aligned}\mathbf{X}\mathbf{w} &= \mathbf{y} \\ \Downarrow \\ \mathbf{X}^{-1}\mathbf{X}\mathbf{w} &= \mathbf{X}^{-1}\mathbf{y} \\ \Downarrow \\ \mathbf{w} &= \mathbf{X}^{-1}\mathbf{y}\end{aligned}$$

Sfortunatamente, questo è vero solo nei casi più banali. $\mathbf{X}$ non è sempre invertibile ed è **necessario un approccio più generale**.

Iniziamo notando che variando $\mathbf{w}$ in realtà stiamo coprendo lo spazio colonna di $\mathbf{X}$.



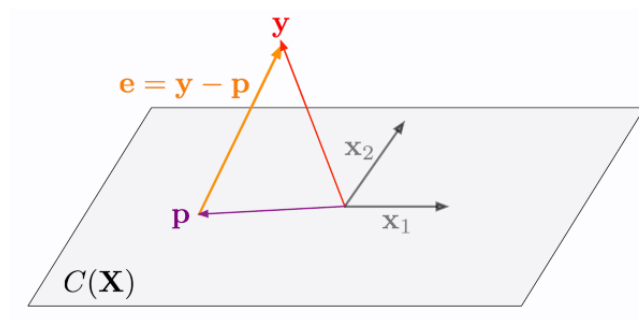
Poiché il sistema di equazioni non è risolubile, ne consegue che non esiste alcuna combinazione di w_i che ci permetta di ottenere y , implicando che y non appartiene a $C(\mathbf{X})$.



Il meglio che possiamo fare è trovare il vettore $\mathbf{w}$ che minimizza la lunghezza dell'errore degli errori $\mathbf{e} = \mathbf{y} - \mathbf{X}\mathbf{w}$.

In altre parole, vogliamo trovare:

$$\arg \min_{\mathbf{w}} \|\mathbf{e}\| = \arg \min_{\mathbf{w}} \|\mathbf{e}\|^2 = \arg \min_{\mathbf{w}} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2$$



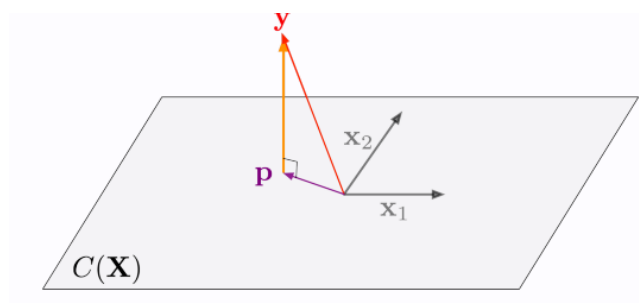
3.2.4 Migliore p

Domanda Come scegliamo il punto migliore sul piano $C(\mathbf{X})$ che si avvicina a $\mathbf{y}$?

Il migliore punto sul piano $C(\mathbf{X})$ che approssima $\mathbf{y}$ è la **proiezione ortogonale** di $\mathbf{y}$ su $C(\mathbf{X})$.

In altre parole, è **sufficiente trovare il punto sul piano $C(\mathbf{X})$ tale che il vettore errore $\mathbf{e}$ sia ortogonale al piano.**

Questo punto corrisponde a $\mathbf{p} = \mathbf{X}\mathbf{w}$, dove $\mathbf{w}$ è il vettore dei pesi ottimali.



Per imporre che un vettore sia ortogonale a un piano, è sufficiente imporre che sia ortogonale a tutti i vettori che attraversano il piano:

$$\mathbf{x}_1 \perp \mathbf{e} \Rightarrow \mathbf{x}_1^T \mathbf{e} = 0$$

$$\mathbf{x}_2 \perp \mathbf{e} \Rightarrow \mathbf{x}_2^T \mathbf{e} = 0$$

...

Che può essere scritto in forma matriciale come:

$$\mathbf{X}^T \mathbf{e} = 0$$

La soluzione può essere trovata risolvendo la seguente equazione:

$$\mathbf{X}^T (\mathbf{y} - \mathbf{X}\mathbf{w}) = 0$$

$$\mathbf{X}^T \mathbf{y} - \mathbf{X}^T \mathbf{X}\mathbf{w} = 0$$

$$\mathbf{X}^T \mathbf{X}\mathbf{w} = \mathbf{X}^T \mathbf{y}$$

$$\mathbf{w} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

Regolarizzazione

Se i dati sono scarsi o rumorosi, la matrice $\mathbf{X}^T \mathbf{X}$ potrebbe essere **mal condizionata** o **singolare**, rendendo problematica l'inversione. In questi casi, è possibile impiegare tecniche di **regolarizzazione** (ad esempio, la regressione di Ridge) per stabilizzare la soluzione.

3.2.5 Soluzione dei minimi quadrati tramite calcolo

Un altro modo per trovare la soluzione dei minimi quadrati è utilizzare il calcolo.

Possiamo riscrivere il problema come:

$$\arg \min_{\mathbf{w}} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2 = \arg \min_{\mathbf{w}} (\mathbf{y} - \mathbf{X}\mathbf{w})^T (\mathbf{y} - \mathbf{X}\mathbf{w}) = \arg \min_{\mathbf{w}} (\mathbf{y}^T \mathbf{y} - 2\mathbf{y}^T \mathbf{X}\mathbf{w} + \mathbf{w}^T \mathbf{X}^T \mathbf{X} \mathbf{w})$$

Derivando rispetto a $\mathbf{w}$ e ponendo la derivata a zero, otteniamo:

$$\frac{\partial}{\partial \mathbf{w}} (\mathbf{y}^T \mathbf{y} - 2\mathbf{y}^T \mathbf{X}\mathbf{w} + \mathbf{w}^T \mathbf{X}^T \mathbf{X} \mathbf{w}) = 0$$

$$-2\mathbf{X}^T \mathbf{y} + 2\mathbf{X}^T \mathbf{X} \mathbf{w} = 0$$

$$\mathbf{X}^T \mathbf{X} \mathbf{w} = \mathbf{X}^T \mathbf{y}$$

$$\mathbf{w} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

Dove usiamo:

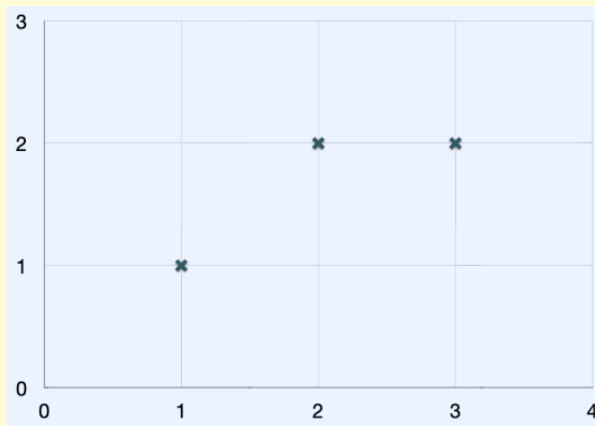
$$\frac{\partial}{\partial \mathbf{w}} [\mathbf{w}^T \mathbf{X}^T \mathbf{X} \mathbf{w}] = 2\mathbf{X}^T \mathbf{X} \mathbf{w}$$

e

$$\frac{\partial}{\partial \mathbf{w}} [\mathbf{y}^T \mathbf{X} \mathbf{w}] = \mathbf{X}^T \mathbf{y}$$

Esempio usando tre punti

$$\mathbf{X} = \begin{bmatrix} 1 & 1 \\ 2 & 1 \\ 3 & 1 \end{bmatrix} \quad \mathbf{y} = \begin{bmatrix} 1 \\ 2 \\ 2 \end{bmatrix}$$



Obiettivo: calcolare $\hat{\mathbf{w}} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$

$$\mathbf{X}^T \mathbf{X} = \begin{bmatrix} 1 & 2 & 3 \\ 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 2 & 1 \\ 3 & 1 \end{bmatrix} = \begin{bmatrix} 14 & 6 \\ 6 & 3 \end{bmatrix}$$

Possiamo calcolare l'inverso di $\mathbf{X}^T \mathbf{X}$ usando il metodo di Gauss-Jordan:

$$\left[\begin{array}{cc|cc} 14 & 6 & 1 & 0 \\ 6 & 3 & 0 & 1 \end{array} \right] \rightarrow \left[\begin{array}{cc|cc} 2 & 0 & 1 & -2 \\ 6 & 3 & 0 & 1 \end{array} \right] \rightarrow \left[\begin{array}{cc|cc} 1 & 0 & \frac{1}{2} & -1 \\ 6 & 3 & 0 & 1 \end{array} \right]$$

$$\left[\begin{array}{cc|cc} 1 & 0 & \frac{1}{2} & -1 \\ 0 & 3 & -3 & 7 \end{array} \right] \rightarrow \left[\begin{array}{cc|cc} 1 & 0 & \frac{1}{2} & -1 \\ 0 & 1 & -1 & \frac{7}{3} \end{array} \right]$$

Esempio usando tre punti

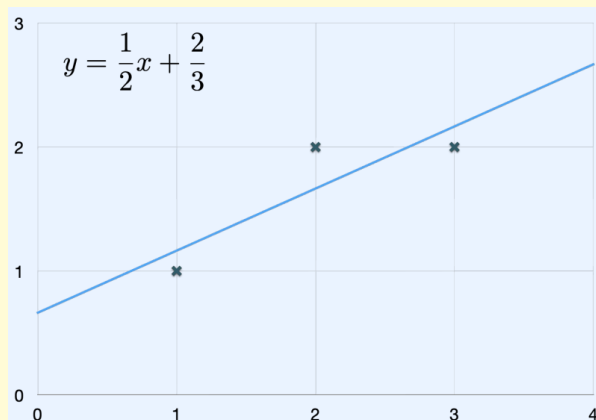
Implicando:

$$(\mathbf{X}^T \mathbf{X}^{-1}) = \begin{bmatrix} \frac{1}{2} & -1 \\ -1 & \frac{7}{3} \end{bmatrix}$$

$$\mathbf{X}^T \mathbf{y} = \begin{bmatrix} 1 & 2 & 3 \\ 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \\ 2 \end{bmatrix} = \begin{bmatrix} 11 \\ 5 \end{bmatrix}$$

La soluzione è quindi:

$$\hat{\mathbf{w}} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y} = \begin{bmatrix} \frac{1}{2} & -1 \\ -1 & \frac{7}{3} \end{bmatrix} \begin{bmatrix} 11 \\ 5 \end{bmatrix} = \begin{bmatrix} \frac{1}{2} \cdot 11 - 1 \cdot 5 \\ -1 \cdot 11 + \frac{7}{3} \cdot 5 \end{bmatrix} = \begin{bmatrix} \frac{1}{2} \\ \frac{2}{3} \end{bmatrix}$$



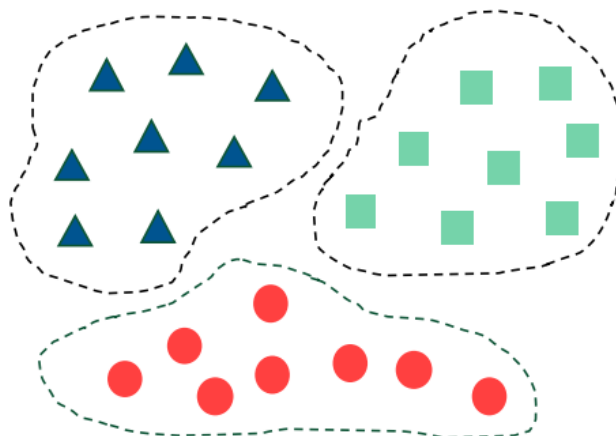
4 Clustering

4.1 Problema del Clustering

È ampiamente studiato e utilizzato nell'apprendimento non supervisionato.

L'obiettivo del Clustering è partizionare il dataset in diversi sottoinsiemi disgiunti (cluster).

Il clustering può essere utilizzato da solo per identificare la struttura intrinseca dei dati, ma può anche fungere da tecnica di pre-elaborazione per altre attività di apprendimento come la classificazione.



4.1.1 Formalizzazione

Dato un dataset $D = \{x_1, x_2, \dots, x_m\}$, contenente m esempi non etichettati, dove ogni esempio $x_i = (x_{i1}, x_{i2}, \dots, x_{in})$ è un vettore n -dimensionale. Poi, un algoritmo di clustering partiziona il dataset D in

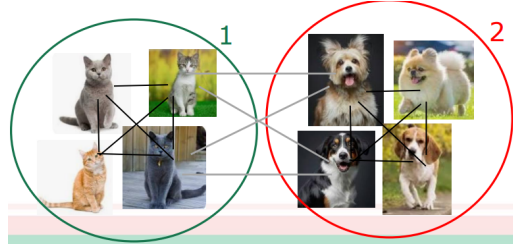
k cluster $\{C_l \mid l = 1, 2, \dots, k\}$, dove $C_l \cap C_{l' \neq l} = \emptyset$ e $D = \bigcup_{l=1}^k C_l$.

Di conseguenza, denotiamo $\lambda_j \in \{1, 2, \dots, k\}$ come l'etichetta degli esempi x_j , (cioè, $x_j \in C_{\lambda_j}$). Quindi il risultato del clustering può essere rappresentato come un vettore di etichette di cluster $\Lambda = (\lambda_1, \lambda_2, \dots, \lambda_m)$ con m elementi.

4.2 Misura delle prestazioni

Le misure di performance per il clustering sono anche chiamate **indici di validità**.

Intuitivamente, vogliamo che le cose di un certo tipo si uniscano; ovvero, **i campioni nello stesso cluster dovrebbero essere il più simili possibile**, mentre i campioni di cluster diversi dovrebbero essere il più diversi possibile. In altre parole, cerchiamo cluster con un'elevata **similarità intracluster** e una **bassa similarità intercluster**.



4.2.1 Misura delle prestazioni per il clustering

External index Confronto del risultato del clustering con un modello di riferimento (etichette fornite dagli esperti).

Internal index Valutare il risultato del clustering senza utilizzare alcun modello di riferimento (ad esempio, varianza, coesione, ecc.)

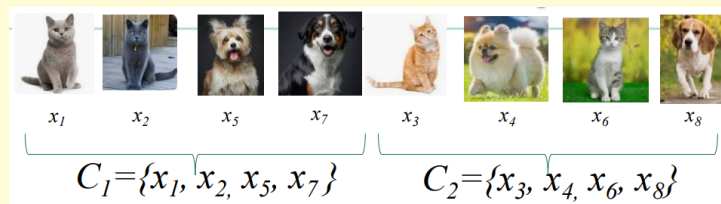
4.2.2 Misura della performance - Indice esterno

Dato un dataset $D = \{x_1, x_2, \dots, x_m\}$, supponiamo che un algoritmo di clustering produca i cluster $C = \{C_1, C_2, \dots, C_k\}$, e un modello di riferimento fornisce i cluster $C^* = \{C_1^*, C_2^*, \dots, C_s^*\}$. Di conseguenza, indichiamo con λ e λ^* i vettori delle etichette di clustering di C e C^* , rispettivamente.

Per ogni coppia di campioni possiamo definire i seguenti quattro termini:

$$\begin{aligned} a &= |SS|, \quad SS = \{(x_i, x_j) \mid \lambda_i = \lambda_j, \lambda_i^* = \lambda_j^*, i < j\} \\ b &= |SD|, \quad SD = \{(x_i, x_j) \mid \lambda_i = \lambda_j, \lambda_i^* \neq \lambda_j^*, i < j\} \\ c &= |DS|, \quad DS = \{(x_i, x_j) \mid \lambda_i \neq \lambda_j, \lambda_i^* = \lambda_j^*, i < j\} \\ d &= |DD|, \quad DD = \{(x_i, x_j) \mid \lambda_i \neq \lambda_j, \lambda_i^* \neq \lambda_j^*, i < j\} \end{aligned} \tag{1}$$

Dove D = diverso cluster stimato, S = stesso cluster stimato.



$$C^* = \{C_1^*, C_2^*\} \quad C_1^* = \{x_1, x_2, x_3, x_6\} \quad C_2^* = \{x_4, x_5, x_7, x_8\}$$

$$\lambda = \{1, 1, 2, 2, 1, 2, 1, 2\} \quad \lambda^* = \{1, 1, 1, 2, 2, 1, 2, 2\}$$

$$a = |SS|, \quad SS = \{(x_i, x_j) \mid \lambda_i = \lambda_j, \lambda_i^* = \lambda_j^*, i < j\}$$

$$(x_1, x_2), (x_4, x_8), (x_3, x_6), (x_5, x_7)$$

G G C C G G C C

Pairs	$\wedge$ S	$\wedge$ D	Total row
True S	4		
True D			
Total col			28

$$\frac{m \cdot m - 1}{2}$$

$$b = |SD|, \quad SD = \{(x_i, x_j) \mid \lambda_i = \lambda_j, \lambda_i^* \neq \lambda_j^*, i < j\}$$

$$(x_1, x_5), (x_1, x_7), (x_2, x_5), (x_2, x_7),$$

G C G C G C G C

$$(x_3, x_4), (x_3, x_8), (x_4, x_6), (x_6, x_8)$$

G C G C G C G C

Pairs	$\wedge$ S	$\wedge$ D	Total row
True S	4		
True D	8		
Total col	12	16	28

Esempio

$$c = |DS|, DS = \{(x_i, x_j) \mid \lambda_i \neq \lambda_j, \lambda_i^* = \lambda_j^*, i < j\}$$

$$\begin{array}{cccccc} (x_1, x_3), & (x_1, x_6), & (x_2, x_3), & (x_2, x_6), \\ G & G & G & G & G & G & G & G \\ (x_4, x_5), & (x_4, x_7), & (x_5, x_8), & (x_7, x_8) \\ C & C & C & C & C & C & C & C \end{array}$$

Pairs	$\wedge$ S	$\wedge$ D	Total row
True S	4	8	12
True D	8		16
Total col	12	16	28

$$d = |DD|, DD = \{(x_i, x_j) \mid \lambda_i \neq \lambda_j, \lambda_i^* \neq \lambda_j^*, i < j\}$$

$$\begin{array}{cccccc} (x_1, x_4), & (x_1, x_8), & (x_2, x_4), & (x_2, x_8), \\ C & G & G & C & G & C & G & C \\ (x_3, x_5), & (x_3, x_7), & (x_5, x_6), & (x_6, x_7) \\ G & C & G & C & C & G & G & C \end{array}$$

Pairs	$\wedge$ S	$\wedge$ D	Total row
True S	4	8	12
True D	8	8	16
Total col	12	16	28

- Jaccard Coefficient, JC:

$$JC = \frac{a}{a + b + c} = \frac{Set_1 \cap Set_2}{Set_1 \cup Set_2}$$

- Fowlkes and Mallows Index, FMI:

$$\text{geometric mean } FMI = \sqrt{\frac{a}{a + b} \cdot \frac{a}{a + c}}$$

- Rand Index, RI:

$$RI = \frac{2(a + b)}{m(m - 1)}$$

Esempio

- Jaccard Coefficient, JC:

$$JC = \frac{a}{a + b + c} = \frac{4}{20} = 0.2$$

- Fowlkes and Mallows Index, FMI:

$$\text{geometric mean } FMI = \sqrt{\frac{a}{a+b} \cdot \frac{a}{a+c}} = \sqrt{\frac{4}{12} \cdot \frac{4}{12}} = \frac{4}{12} = 0.33$$

- Rand Index, RI:

$$RI = \frac{2(a+b)}{m(m-1)} = \frac{12}{28} = 0.428$$

Pairs	$\wedge$ S	$\wedge$ D	Total row
True S	4	8	12
True D	8	8	16
Total col	12	16	28

4.2.3 Misura della performance - Indice interno

Cluster Cohesion: Misura quanto strettamente correlati sono gli oggetti in un cluster

- Esempio: Sum Squared Error (SSE)
- Cohesion è misurato da **Within cluster Sum of Squares (WCSS):**

$$WSS = \sum_i \sum_{x \in C_i} (x - m_i)^2$$

Più piccolo è, meglio è.

Cluster Separation: Misura quanto un cluster è distinto o ben separato dagli altri cluster

- Separation è misurata da **Between cluster Sum of Squares (BSS):**

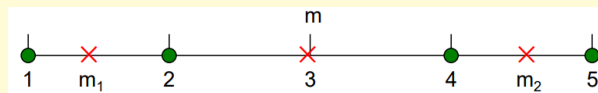
$$BSS = \sum_i |C_i| (m - m_i)^2$$

– dove $|C_i|$ è la dimensione del cluster i .

Più grande è, meglio è.

Esempio - SSE

$$BSS + WSS = \text{constant}$$



K=1 cluster:

- $WSS = (1 - 3)^2 + (2 - 3)^2 + (4 - 3)^2 + (5 - 3)^2 = 10$
- $BSS = 4 \times (3 - 3)^2 = 0$
- $Total = 10 + 0 = 10$

K=2 clusters:

- $WSS = (1 - 1.5)^2 + (2 - 1.5)^2 + (4 - 4.5)^2 + (5 - 4.5)^2 = 1$
- $BSS = 2 \times (3 - 1.5)^2 + 2 \times (4.5 - 3)^2 = 9$
- $Total = 1 + 9 = 10$

R²: un modo per combinarli

$$R^2 = 1 - \frac{WSS}{SSE}$$

Più grande è, meglio è.

L'indice R-quadrato misura la **proporzione della varianza totale (SSE) nei dati che è spiegata dalla soluzione di clustering (WSS)**, dove la varianza totale (SSE) è la varianza dell'intero set di dati, e la varianza all'interno dei cluster (WSS) è la somma delle varianze di ciascun cluster.

Dati i cluster generati $C = \{C_1, C_2, \dots, C_k\}$, definiamo i seguenti quattro termini:

- La distanza media tra i campioni nel cluster C :

$$avg(C) = \frac{2}{|C|(|C| - 1)} \sum_{1 \leq i < j \leq |C|} dist(x_i, x_j)$$



- La distanza più grande tra i campioni nel cluster C :

$$diam(C) = \max_{1 \leq i < j \leq |C|} dist(x_i, x_j)$$



- La distanza tra due campioni più vicini nei cluster C_i e C_j :

$$d_{min}(C) = \min_{x_i \in C_i, x_j \in C_j} dist(x_i, x_j)$$



- La distanza tra i centroidi dei cluster C_i e C_j :

$$d_{cen}(C) = dist(\mu_i, \mu_j)$$



- Davies-Bouldin Index, DBI:**

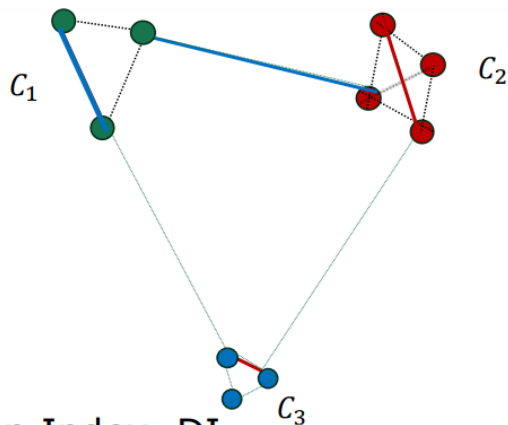
$$DBI = \frac{1}{k} \sum_{i=1}^k \max_{j \neq i} \left(\frac{avg(C_i) + avg(C_j)}{d_{cen}(C_i, C_j)} \right)$$



Più piccolo è, meglio è.

- Dunn Index, DI:**

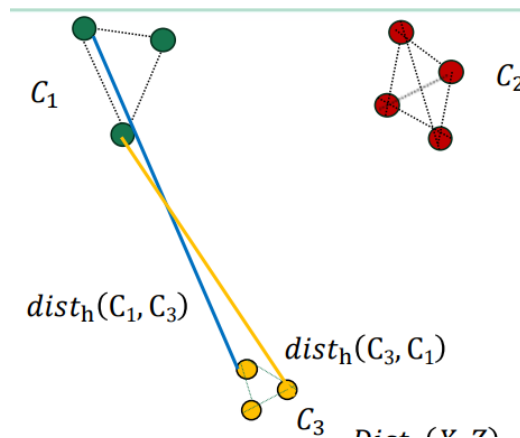
$$DI = \min_{1 \leq i \leq k} \left\{ \min_{j \neq i} \left(\frac{d_{min}(C_i, C_j)}{\max_{1 \leq l \leq k} diam(C_l)} \right) \right\}$$



Più grande è, meglio è.

- **Hausdorff distance:**

$$Dist_h(X, Z) = \max\{dist_h(X, Z), dist_h(Z, X)\} \quad \text{dove} \quad dist_h(X, Z) = \max_{x \in X} \min_{z \in Z} \|x - z\|_2$$



Più grande è, meglio è.

Un'equivalenza utile

$$avg(C) = \frac{2}{|C|(|C| - 1)} \sum_{1 \leq i < j \leq |C|} dist(x_i, x_j)$$

Consideriamo ogni coppia di oggetti solo una volta.

$$Dis(C) = \frac{1}{|C|^2} \sum_{x_i \in C} \sum_{x_j \in C} [x_i - x_j]^2 = 2 \cdot Var(C)$$

Consideriamo ogni coppia di oggetti due volte (con gli oggetti in ordine inverso) con:

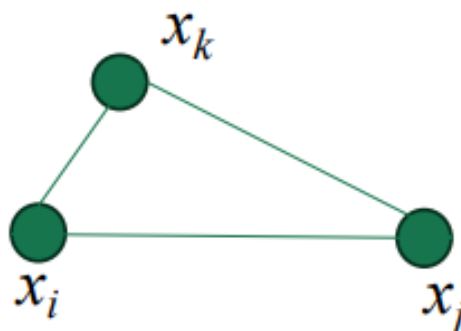
$$Var(C) = \frac{1}{|C|} \sum_{x \in C} (x - \bar{x})^2 \quad \bar{x} = \frac{1}{|C|} \sum_{x \in C} x$$

È stato dimostrato che $\bar{x}$ è la posizione che minimizza la somma del quadrato della distanza euclidea con i punti di un cluster.

4.3 Calcolo della distanza

Gli assiomi della misura della distanza:

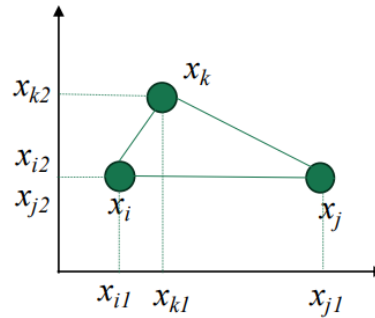
- Non negatività: $dist(x_i, x_j) \geq 0$
- Identità degli indiscernibili: $dist(x_i, x_j) = 0 \Leftrightarrow x_i = x_j$
- Simmetria: $dist(x_i, x_j) = dist(x_j, x_i)$
- Subadditività (Disuguaglianza triangolare): $dist(x_i, x_j) \leq dist(x_i, x_k) + dist(x_k, x_j)$



Una misurazione che viene comunemente utilizzata è **Minkowski distance**:

$$dist(x_i, x_j) = \left(\sum_{u=1}^n |x_{iu} - x_{ju}|^p \right)^{1/p}$$

- Per $p = 1$, otteniamo la **distanza di Manhattan** (o distanza L1)
- Per $p = 2$, otteniamo la **distanza euclidea** (o distanza L2)
La distanza euclidea è una generalizzazione del teorema di Pitagora.



4.3.1 Attributi

Attributo continuo avente domini infiniti (può essere manipolato da qualsiasi matematica)

Attributo ordinale (può essere ordinato o differenziato)

Attributo categoriale (possono essere enumerati o distinti) aventi domini finiti

Ad esempio, un attributo categoriale con dominio $\{1, 2, 3\}$ è più simile a un attributo ordinale o continuo poiché la distanza può essere calcolata con i valori dell'attributo, ovvero 1 è più vicino a 2 che a 3.

Attributo non ordinale (o nominale) Se il dominio di un attributo è discreto, come $\{aereo, treno, nave\}$, allora la distanza non può essere calcolata direttamente con i valori dell'attributo, e tali attributi sono attributi non ordinali.

4.3.2 Value Difference Metric, VDM

Sia $m_{u,a}$ il numero di campioni che assumono valore a sull'attributo u , e $m_{u,a,i}$ il numero di campioni all'interno dell' i -esimo cluster che assumono valore a sull'attributo u , e k è il numero di cluster. Quindi la distanza VDM tra due valori categoriali a e b dell'attributo u è:

$$VDM_p(a, b) = \sum_{i=1}^k \left| \frac{m_{u,a,i}}{m_{u,a}} - \frac{m_{u,b,i}}{m_{u,b}} \right|^p$$

4.3.3 Misura della distanza con tipi di attributi misti

MinkovDM_p

$$MinkovDM_p(\vec{x}_i, \vec{x}_j) = \left(\sum_{u=1}^{n_c} |x_{iu} - x_{ju}|^p + \sum_{u=n_c+1}^n VDM_p(x_{iu}, x_{ju}) \right)^{\frac{1}{p}}$$

$n_c \rightarrow$ numero di attributi continui.

$n \rightarrow$ numero totale di attributi categoriali.

Distanza pesata Quando attributi diversi hanno importanza diversa:

$$dist(\vec{x}_i, \vec{x}_j) = (\omega_1 \cdot |x_{i1} - x_{j1}|^p + \omega_2 \cdot |x_{i2} - x_{j2}|^p + \dots + \omega_n \cdot |x_{in} - x_{jn}|^p)^{\frac{1}{p}}$$

$$\omega_i \geq 0, \sum_{i=1}^n \omega_i = 1$$

4.4 Prototype Clustering

Prototype Clustering, noto anche come **clustering basato su prototipi**, presuppone che la struttura del clustering possa essere rappresentata con un **insieme di prototipi**.

Algoritmi In genere, tali algoritmi partono da alcuni prototipi iniziali, per poi aggiornarli e ottimizzarli iterativamente.

4.4.1 K-means Clustering

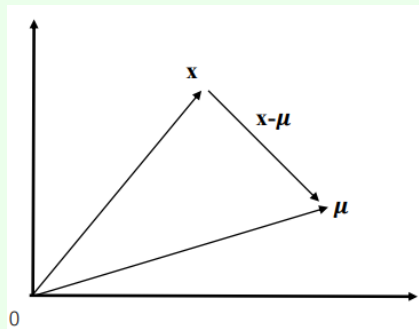
Dato un dataset $D = \{x_1, x_2, \dots, x_m\}$, l'obiettivo del K-means è quello di minimizzare l'errore quadratico dei cluster $C = \{C_1, C_2, \dots, C_k\}$:

$$E = \sum_{i=1}^k \sum_{x \in C_i} \|x - \mu_i\|_2^2$$

dove μ_i è il vettore medio del cluster C_i .

Intuitivamente, E rappresenta la vicinanza tra il vettore medio di un cluster e i campioni all'interno di quel cluster, dove un valore E più piccolo indica una maggiore similarità intra-cluster.

The norm-2 of a vector



$$\mathbf{X} = \begin{bmatrix} X_1 \\ X_2 \\ \vdots \\ X_n \end{bmatrix} \quad \mu = \begin{bmatrix} \mu_1 \\ \mu_2 \\ \vdots \\ \mu_n \end{bmatrix} \quad \mathbf{X} - \mu = \begin{bmatrix} X_1 - \mu_1 \\ X_2 - \mu_2 \\ \vdots \\ X_n - \mu_n \end{bmatrix}$$

$$\|x - \mu\|_2 = \sqrt{(x_1 - \mu_1)^2 + (x_2 - \mu_2)^2 + \dots + (x_n - \mu_n)^2}$$

$$\|x - \mu\|_2^2 = (x_1 - \mu_1)^2 + (x_2 - \mu_2)^2 + \dots + (x_n - \mu_n)^2 = \sum_{i=1}^n (x_i - \mu_i)^2$$

$$\|x - \mu\|_2^2 = (\mathbf{X} - \mu)^T (\mathbf{X} - \mu) \quad (\text{in notazione vettoriale})$$

Algoritmo (ottimizzazione iterativa)

- inizializza i vettori medi dei cluster ripeti

1. (aggiorna) i cluster
 2. calcola i nuovi vettori medi dei cluster
- finché i cluster non cambiano

Algorithm 1 k-means Clustering

Require: Data set $D = \{x_1, x_2, \dots, x_m\}$; Number of clusters k .

```

1: Process:
2: Randomly select  $k$  samples as the initial mean vectors  $\{\mu_1, \mu_2, \dots, \mu_k\}$ .
3: repeat
4:    $C_i = \emptyset$  ( $1 \leq i \leq k$ )
5:   for  $j = 1, 2, \dots, m$  do
6:     Compute the distance between sample  $x_j$  and each mean vector  $\mu_i$  ( $1 \leq i \leq k$ ):
        $d_{ji} = \|x_j - \mu_i\|_2$ 
7:     According to the nearest mean vector, decide the cluster label of  $x_j$ :  $\lambda_j =$ 
        $\arg \min_{i \in \{1, 2, \dots, k\}} d_{ji}$ 
8:     Move  $x_j$  to the corresponding cluster:  $C_{\lambda_j} = C_{\lambda_j} \cup \{x_j\}$ 
9:   end for
10:  for  $i = 1, 2, \dots, k$  do
11:    Compute the updated mean vectors:  $\mu'_i = \frac{1}{|C_i|} \sum_{x \in C_i} x$ 
12:    if  $\mu'_i \neq \mu_i$  then
13:      Update the current mean vector  $\mu_i$  to  $\mu'_i$ 
14:    else
15:      Leave the current mean vector unchanged
16:    end if
17:  end for
18: until All mean vectors remain unchanged

```

Ensure: Clusters $\mathcal{C} = \{C_1, C_2, \dots, C_k\}$.

Un esempio per l'algoritmo k-means

Prendiamo il set di dati 4:0 dell'anguria nella tabella seguente come esempio per dimostrare l'algoritmo k-means. Per semplicità di discussione, rappresentiamo il campione con l'ID i come x_i .

ID	density	sugar	ID	density	sugar	ID	density	sugar
1	0.697	0.460	11	0.245	0.057	21	0.748	0.232
2	0.774	0.376	12	0.343	0.099	22	0.714	0.346
3	0.634	0.264	13	0.639	0.161	23	0.483	0.312
4	0.608	0.318	14	0.657	0.198	24	0.478	0.437
5	0.556	0.215	15	0.360	0.370	25	0.525	0.369
6	0.403	0.237	16	0.593	0.042	26	0.751	0.489
7	0.481	0.149	17	0.719	0.103	27	0.532	0.472
8	0.437	0.211	18	0.359	0.188	28	0.473	0.376
9	0.666	0.091	19	0.339	0.241	29	0.725	0.445
10	0.243	0.267	20	0.282	0.257	30	0.446	0.459

Supponendo di impostare $k = 3$, l'algoritmo seleziona causalmente tre campioni x_6, x_{12}, x_{24} come vettori medi iniziali dei cluster, cioè, $\mu_1 = (0.403; 0.237)$, $\mu_2 = (0.343; 0.099)$, $\mu_3 = (0.478; 0.437)$

Poi, per il campione $x_1 = (0.697; 0.460)$, la sua distanza dai tre vettori medi μ_1, μ_2, μ_3 è 0.369, 0.506, e 0.220, rispettivamente. Quindi, x_1 viene assegnato al cluster C_3 . Similmente, valutiamo tutti i campioni del dataset e troviamo i seguenti cluster:

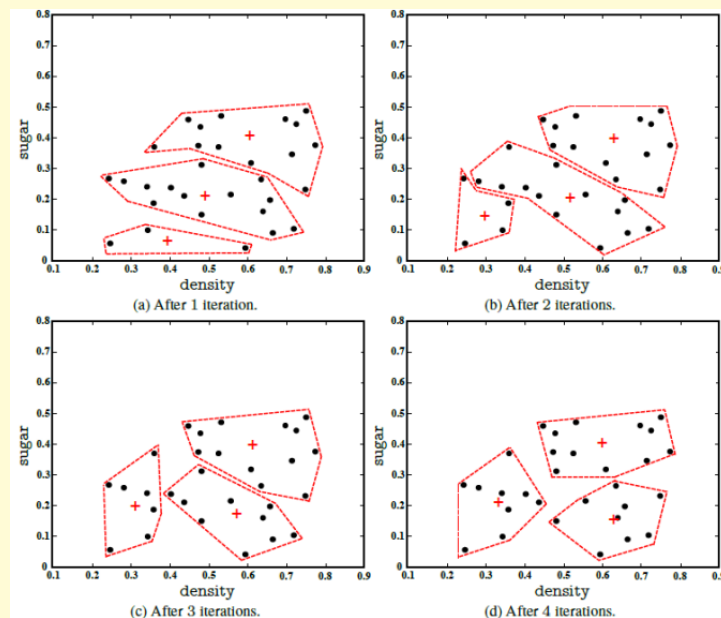
$$C_1 = \{x_3, x_5, x_6, x_7, x_8, x_9, x_{10}, x_{13}, x_{14}, x_{17}, x_{18}, x_{19}, x_{20}, x_{23}\}$$

$$C_2 = \{x_{11}, x_{12}, x_{16}\}$$

$$C_3 = \{x_1, x_2, x_4, x_{15}, x_{21}, x_{22}, x_{24}, x_{25}, x_{26}, x_{27}, x_{28}, x_{29}, x_{30}\}$$

$$\mu'_1 = (0.493; 0.207), \mu'_2 = (0.394; 0.066), \mu'_3 = (0.602; 0.396)$$

Risultati dell'algoritmo k-means



4.4.2 Learning Vector Quantization (LVQ)

A differenza del tipico algoritmo di clustering, LVQ presuppone che i **campioni di dati siano etichettati** e che il **processo di clustering sia assistito da informazioni supervisionate**.

Dato un dataset $D = \{(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)\}$, LVQ mira a imparare un set n-dimensionale di vettori di prototipi $\{p_1, p_2, \dots, p_q\}$ dove ogni vettore di prototipi rappresenta un cluster.

Algorithm 2 Learning Vector Quantization (LVQ)

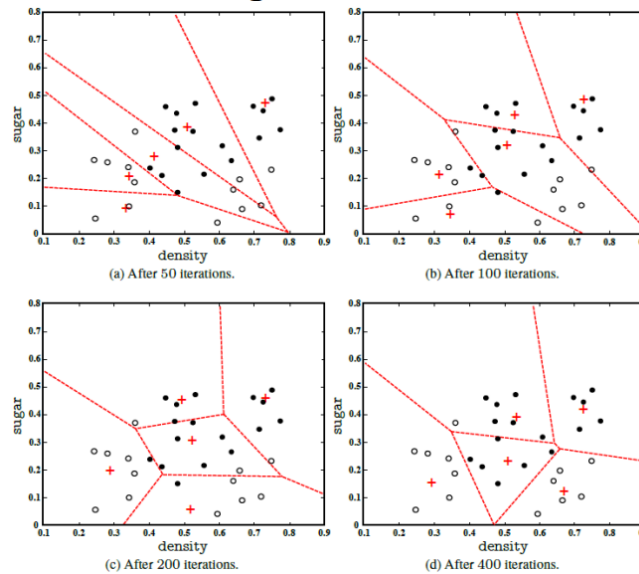
Require: Training set $D = \{(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)\}$; Number of clusters q . Initial labels of prototype vectors $\{t_1, t_2, \dots, t_q\}$. Learning rate η .

- 1: **Process:**
- 2: Initialize a set of prototype vectors $\{p_1, p_2, \dots, p_q\}$;
- 3: **repeat**
- 4: Randomly pickup a sample (x_j, y_j) from the data set D ;
- 5: Compute the distance between x_j and $p_i (1 \leq i \leq q)$: $d_{ji} = \|x_j - p_i\|_2$;
- 6: Find the nearest prototype vector p_{i^*} for x_j : $i^* = \arg \min_{i \in \{1, 2, \dots, q\}} d_{ji}$;
- 7: **if** $y_j = t_{i^*}$ **then**
- 8: $p' = p_{i^*} + \eta \cdot (x_j - p_{i^*})$; // p is moved toward x_j (their distance is decreased by η)
- 9: **else**
- 10: $p' = p_{i^*} - \eta \cdot (x_j - p_{i^*})$; // p is moved away from x_j (their distance is increased by η)
- 11: **end if**
- 12: Update the prototype vector p_{i^*} to p' ;
- 13: **until** The termination condition is met

Ensure: Prototype vectors $\{p_1, p_2, \dots, p_q\}$.

$$\|p' - x_j\|_2 = \|p_{i^*} + \eta \cdot (x_j - p_{i^*}) - x_j\|_2 = (1 - \eta) \cdot \|p_{i^*} - x_j\|_2$$

Risultati del Clustering

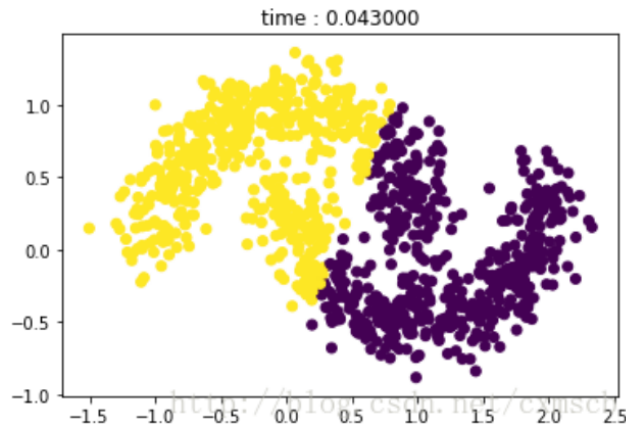


Un vettore prototipo p_i definisce una regione R_i , dove la distanza da qualsiasi campione al suo interno a p_i non è maggiore della distanza da qualsiasi altro vettore prototipo. Le regioni formano una **tassellatura di Voronoi**.

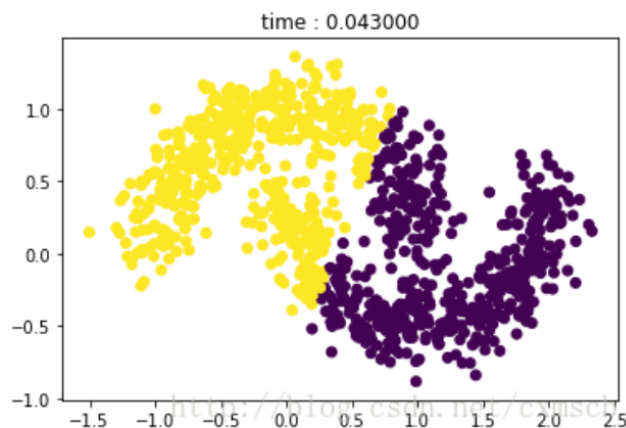
Se tutti i campioni sono rappresentati dai vettori prototipo, allora abbiamo effettuato una **compressione con perdita di dati**: ecco perché si chiama **quantizzazione vettoriale**.

4.5 Density Clustering

Applicando il clustering k-means ai dati seguenti, risulta difficile per il clustering basato su prototipi trovare le informazioni sulla densità, il che porta a risultati ulteriori rispetto a quelli attesi:



Clustering basato sulla densità: valutare la connettività tra i campioni dal punto di vista della densità ed espandere i cluster aggiungendo campioni collegabili.



4.5.1 Definizione di clustering di densità

Il **clustering di densità** è anche noto come clustering basato sulla densità. Supponendo che la struttura del clustering possa essere determinata dalle densità delle distribuzioni campionarie.

In genere, gli algoritmi di clustering di densità **valutano la connettività tra i campioni dal punto di vista della densità**. Espandono i cluster aggiungendo campioni collegabili.

Presentiamo **DBSCAN**, un algoritmo rappresentativo di clustering di densità.

4.5.2 DBSCAN

L'algoritmo DBSCAN caratterizza la densità delle distribuzioni campionarie mediante una **coppia di parametri di vicinato** (ϵ , $MinPts$).

ϵ – neighborhood: per $x_j \in D$, il suo ϵ – neighborhood include tutti i campioni in D la cui distanza da x_j non è maggiore di ϵ ;

Core object: se l' ϵ – neighborhood di un campione x_j contiene almeno $MinPts$ campioni, allora x_j è un core object;

Directly density-reachable: x_j è detto directly density-reachable da x_i se x_i è un core object e x_j è all'interno dell' ϵ – neighborhood di x_i ;

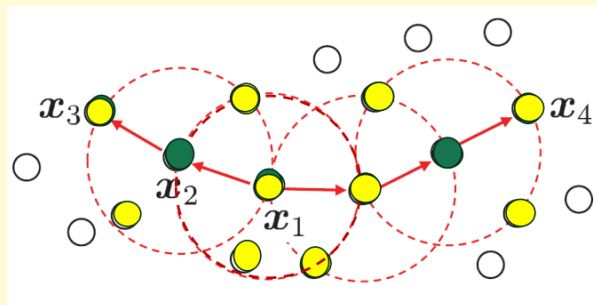
Density-reachable: x_j è detto density-reachable da x_i se esiste una catena di campioni $p_1, p_2, \dots, p_n$ dove $p_1 = x_i$, $p_n = x_j$, e p_{i+1} è directly density-reachable da p_i ;

Density-connected: x_i e x_j sono detti density-connected se esiste un campione x_k tale che entrambi sono density-reachable da x_k .

Esempio

Sia $MinPts = 3$.

I cerchi tratteggiati mostrano l' ϵ -neighborhood di ciascun campione.



- x_1 è un core object
- x_2 è directly density-reachable da x_1
- x_3 è density-reachable da x_1 tramite x_2
- x_3 e x_4 sono density-connected

Definizione di un cluster Il più grande insieme di campioni connessi in base alla densità, derivati da relazioni raggiungibili in base alla densità.

Formalizzazione Dati i parametri di vicinato, un cluster è un sottoinsieme non vuoto con le seguenti proprietà:

- **Connectivity:** $x_i \in C, x_j \in C \Rightarrow x_i$ e x_j sono density-connected
- **Maximality:** $x_i \in C, x_j$ è density-reachable da $x_i \Rightarrow x_j \in C$

In realtà:

1. Se x è un core object e
2. sia $X = \{x' \in D \mid x' \text{ è density-reachable da } x\}$ denota l'insieme dei campioni density-reachable da x ,

allora si può dimostrare che X è un cluster che soddisfa sia la connettività che la massimalità.

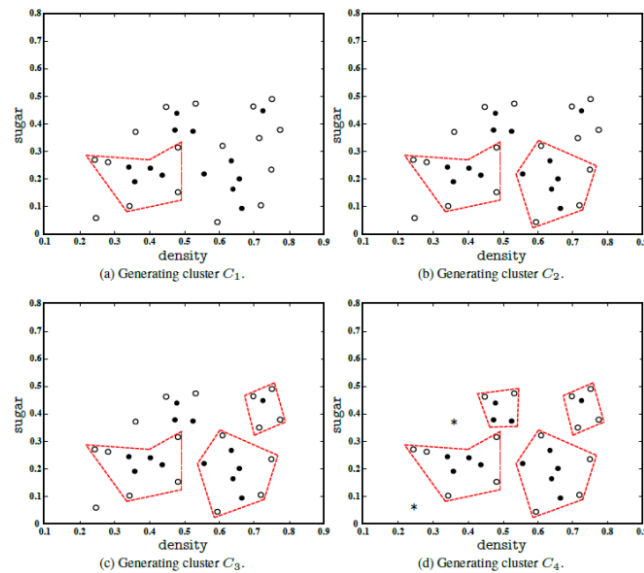
Risultati del Clustering

Algorithm 3 DBSCAN

Require: Data set $D = \{x_1, x_2, \dots, x_m\}$; Neitghborhood parameters $(\epsilon, MinPts)$.

```
1: Process:
2: Initialize the set of core objects:  $\Omega = \emptyset$ ;
3: for  $j = 1, 2, \dots, m$  do
4:   Determine the  $\epsilon$  - neighborhood  $N_\epsilon(x_j)$  of sample  $x_j$ ;
5:   if  $|N_\epsilon(x_j)| \geq MinPts$  then
6:     Add  $x_j$  to the set of core objects:  $\Omega = \Omega \cup \{x_j\}$ ;
7:   end if
8: end for
9: Initialize the number of clusters:  $k = 0$ ;
10: Initialize the set of unprocessed samples:  $\Gamma = D$ ;
11: while  $\Gamma \neq \emptyset$  do
12:   Keep a copy of the current unprocessed data set:  $\Gamma_{old} = \Gamma$ ;
13:   Randomly select a core object  $o \in \Omega$ , and initialize the queue  $Q = \langle o \rangle$ ;
14:    $\Gamma = \Gamma \setminus \{o\}$ ;
15:   while  $Q \neq \emptyset$  do
16:     Dequeue the first sample  $q$  from  $Q$ ;
17:     if  $|N_\epsilon(q)| \geq MinPts$  then
18:       Letting  $\Delta = N_\epsilon(q) \cap \Gamma$ ;
19:       Enqueue the samples in  $\Delta$  into  $Q$ ;
20:        $\Gamma = \Gamma \setminus \Delta$ ; //  $\Gamma$  contains examples not in the next cluster
21:     end if
22:   end while
23:    $k = k + 1$  generate cluster  $C_k = \Gamma_{old} \setminus \Gamma$ ; //  $\Gamma_{old}$  by a double subtraction satisfies the maximality
24:    $\Omega = \Omega \setminus C_k$ ;
25: end while
```

Ensure: Clusters $\mathcal{C} = \{C_1, C_2, \dots, C_k\}$.

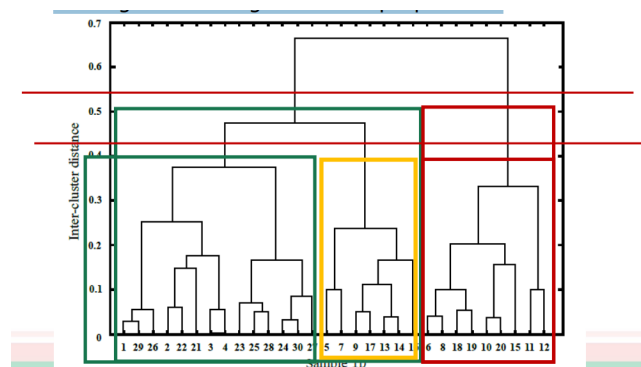


4.6 Hierarchical Clustering

Il clustering gerarchico mira a creare una **struttura di clustering ad albero** dividendo un set di dati in diversi livelli. La gerarchia dei cluster può essere formata adottando una strategia bottom-up o top-down.

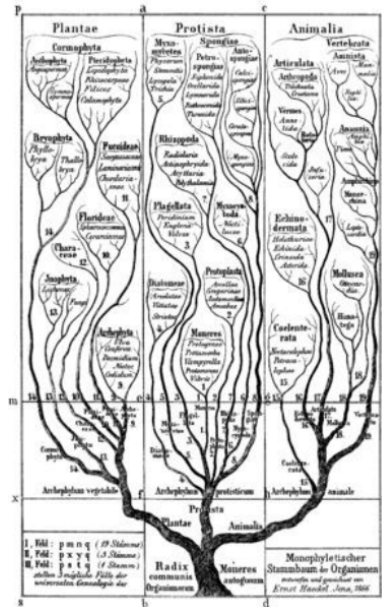
4.6.1 Punti di forza del clustering gerarchico

Non è necessario presupporre un numero specifico di cluster. È possibile ottenere qualsiasi numero desiderato di cluster "tagliando" il dendrogramma al livello appropriato.



Possono corrispondere a tassonomie significative.

Esempio nelle scienze biologiche (ad esempio, regno animale, ricostruzione filogenetica, ...).



4.6.2 AGNES

Algoritmo AGNES (clustering gerarchico **bottom-up**)

- inizia considerando ogni campione nel set di dati come un cluster iniziale.
- Quindi, a ogni round, due cluster più vicini vengono uniti per formare un nuovo cluster.
- Questo processo si ripete finché il numero di cluster non raggiunge il valore predefinito.

Definiamo le distanze di cluster dati C_i e C_j in forme diverse.

Distanza minima:

$$d_{min}(C_i, C_j) = \min_{x \in C_i, z \in C_j} dist(x, z)$$

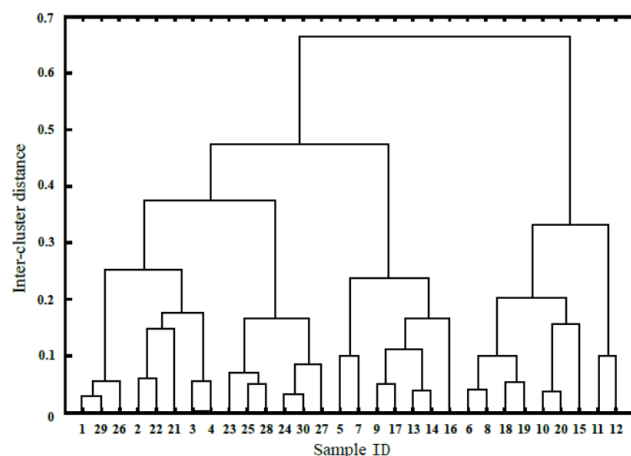
Distanza massima:

$$d_{max}(C_i, C_j) = \max_{x \in C_i, z \in C_j} dist(x, z)$$

Distanza media:

$$d_{avg}(C_i, C_j) = \frac{1}{|C_i| \cdot |C_j|} \sum_{x \in C_i} \sum_{z \in C_j} dist(x, z)$$

Dendrogramma



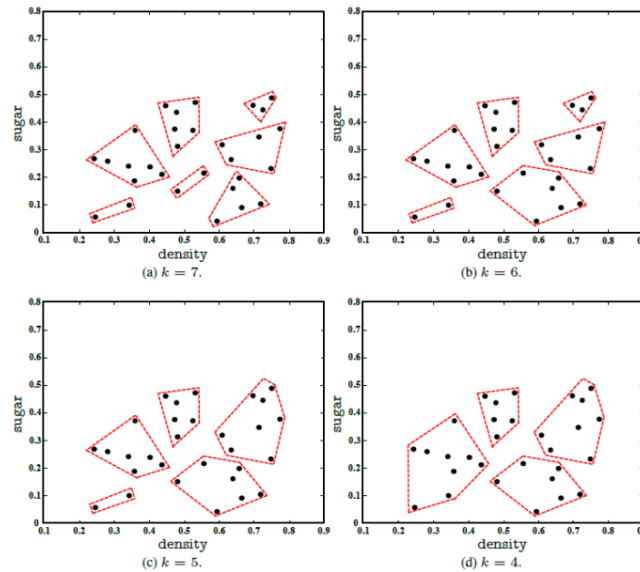
Algorithm 4 AGNES

Require: Data set $D = \{x_1, x_2, \dots, x_m\}$; Cluster distance metric function d ; Number of clusters k .

```
1: Process:
2: for  $j = 1, 2, \dots, m$  do
3:    $C_j = \{x_j\}$ ; //Starts with singleton clusters
4: end for
5: for  $i = 1, 2, \dots, m$  do
6:   for  $j = i + 1, \dots, m$  do
7:      $M(i, j) = d(C_i, C_j)$  //Matrix of point to point distances (symmetric)
8:      $M(j, i) = M(i, j)$ 
9:   end for
10: end for
11: Set the current number of clusters:  $q = m$ ; //Actual number of tree levels
12: while  $q > k$  do //Until the number of levels is reached
13:   Find two clusters  $C_{i^*}$  e  $C_{j^*}$  that have shortest distance;
14:   Merge  $C_{i^*}$  e  $C_{j^*}$ :  $C_{i^*} = C_{i^*} \cup C_{j^*}$ ;
15:   for  $j = j^* + 1, j^* + 2, \dots, q$  do
16:     Change the index of  $C_j$  to  $C_{j-1}$ ; //Shifts clusters indexes so that cluster  $j$  will be forgotten
17:   end for
18:   Delete the  $j^*$ th row and  $j^*$ th column of the distance matrix  $M$ ;
19:   for  $j = 1, 2, \dots, q - 1$  do
20:      $M(i^*, j) = d(C_{i^*}, C_j)$  //Updates the distance matrix considering the presence of the new
    cluster  $i^*$  and all other clusters  $j$ 
21:      $M(j, i^*) = M(i^*, j)$ 
22:   end for
23:    $q = q - 1$ ;
24: end while
```

Ensure: Clusters $\mathcal{C} = \{C_1, C_2, \dots, C_k\}$.

4.6.3 Risultati del Clustering



4.7 Probabilistic Clustering

4.7.1 Prototype Clustering - Mixture of-Gaussian Clustering

A differenza di k-means e LVQ, il clustering misto di gaussiane **non utilizza vettori prototipo**, ma **modelli probabilistici** per rappresentare le strutture di clustering.

Definizione di distribuzione gaussiana multivariata Per un vettore casuale x in uno spazio campione n -dimensionale χ ,

$$p(x) = \frac{1}{(2\pi)^{n/2} |\Sigma|^{1/2}} e^{-\frac{1}{2}(x-\mu)^T \Sigma^{-1} (x-\mu)}$$

dove μ è il vettore n -dimensionale della media e Σ è la matrice di covarianza $n \times n$. Scriviamo la funzione di densità di probabilità come $p(x | \mu, \Sigma)$.

4.7.2 Calcolo della matrice di covarianza

Matrice di dati X (m oggetti, n features)

$$\mathbf{X} = \begin{bmatrix} x_{11} & \cdots & x_{1n} \\ \vdots & \ddots & \vdots \\ x_{m1} & \cdots & x_{mn} \end{bmatrix}$$

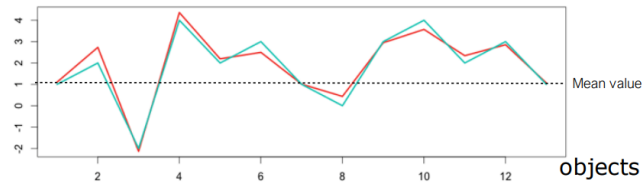
Matrice di covarianza $\Sigma (n \times n)$ L'elemento σ_{lj} è la covarianza tra le features l e j su tutti gli oggetti: misura quanto fortemente le features l e j variano insieme negli m oggetti.

Se $l = j$ è la varianza della caratteristica l .

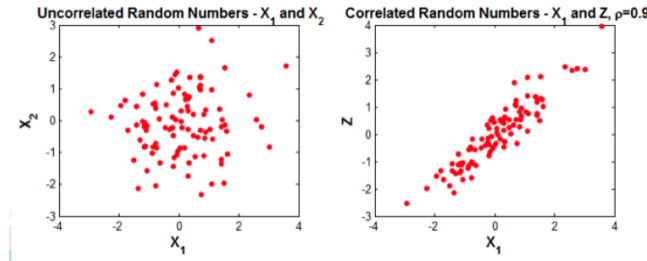
$$\Sigma = \begin{bmatrix} \sigma_{11} & \cdots & \sigma_{1n} \\ \vdots & \ddots & \vdots \\ \sigma_{1n} & \cdots & \sigma_{nn} \end{bmatrix} \quad \sigma_{lj} = \frac{1}{m} \sum_{k=1}^m (x_{kl} - \bar{x}_l)(x_{kj} - \bar{x}_j)$$

4.7.3 Mostrare la correlazione tra due features

Caratteristiche positivamente correlate (valori dei punti nelle linee rosse e verdi)



Scatter plots:



$$\rho = \frac{\sigma_{12}}{\sqrt{\sigma_{11} \cdot \sigma_{22}}}$$

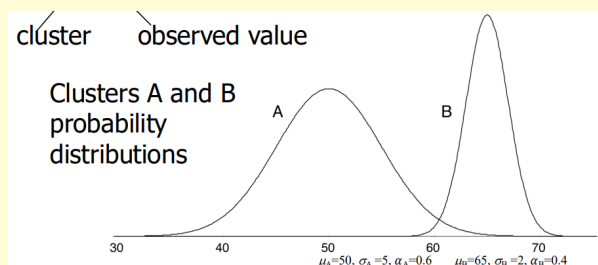
4.7.4 Definizione della distribuzione mista di Gaussiane

$$p_M(x) = \sum_{i=1}^k \alpha_i p(x | \mu_i, \Sigma_i)$$

Consiste di k mixture components, ogni componente corrisponde a una distribuzione gaussiana. μ_i e Σ_i sono i parametri della i -esimo mixture component. $\alpha_i > 0$ sono i coefficienti di mixture corrispondenti, dove $\sum_{i=1}^k \alpha_i = 1$ (la mixture è finita).

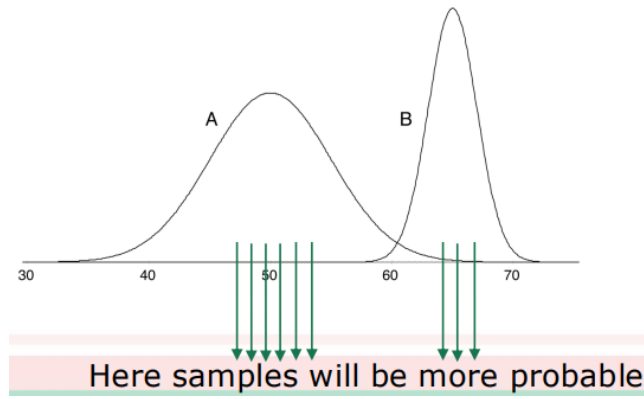
Un modello di mixture a due cluster

data											
A	51	B	62	B	64	A	48	A	39	A	51
A	43	A	47	A	51	B	64	B	62	A	48
B	62	A	52	A	52	A	51	B	64	B	64
B	64	B	64	B	62	B	63	A	52	A	42
A	45	A	51	A	49	A	43	B	63	A	48
A	42	B	65	A	48	B	65	B	64	A	41
A	46	A	48	B	62	B	66	A	48		
A	45	A	49	A	43	B	65	B	64		
A	45	A	46	A	40	A	46	A	48		
model											



Supponiamo che i campioni siano generati da una distribuzione mista di Gaussiane:

1. Innanzitutto, seleziona i componenti della mixture gaussiana utilizzando la distribuzione a priori definita da $\alpha_1, \alpha_2, \dots, \alpha_k$, dove α_i è la probabilità di selezionare il i -esimo componente della mixture.
2. Quindi, genera campioni campionando dalle funzioni di densità di probabilità dei componenti della mixture selezionati.



4.7.5 Ottimizzazione dei parametri del modello

Massima likelihood

$$p_M(x) = \sum_{i=1}^k \alpha_i p(x | \mu_i, \Sigma_i)$$

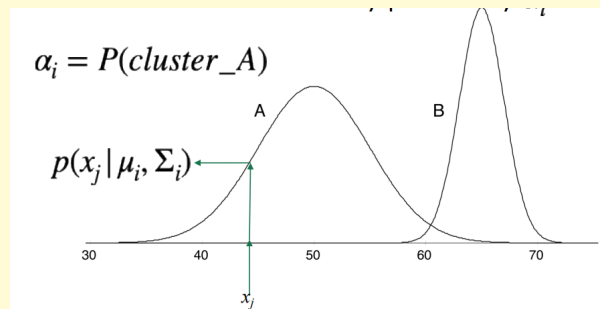
$$LL(D) = \ln \left(\prod_{j=1}^m p_M(x_j) \right) = \sum_{j=1}^m \ln \left(\sum_{i=1}^k \alpha_i \cdot p(x_j | \mu_i, \Sigma_i) \right)$$

$$\frac{\partial LL(D)}{\partial \mu_i} = 0 \quad \Rightarrow \quad \mu_i = \frac{\sum_{j=1}^m \gamma_{ji} x_j}{\sum_{j=1}^m \gamma_{ji}}$$

$$\gamma_{ji} = p_M(z_j = i | x_j) (1 \leq i \leq k); \quad \gamma_{ji} = \frac{\alpha_i \cdot p(x_j | \mu_i, \Sigma_i)}{\sum_{l=1}^k \alpha_l \cdot p(x_j | \mu_l, \Sigma_l)}$$

Esempio

Dato il cluster A , viene selezionato in base alla probabilità α_i :



$$\gamma_{ji} = \frac{\alpha_i \cdot p(x_j | \mu_i, \Sigma_i)}{\sum_{l=1}^k \alpha_l \cdot p(x_j | \mu_l, \Sigma_l)}$$

$$\mu_i = \frac{\sum_{j=1}^m \gamma_{ji} x_j}{\sum_{j=1}^m \gamma_{ji}}$$

Sia:

$$\frac{\partial LL(D)}{\partial \Sigma_i} = 0 \quad \Rightarrow \quad \Sigma_i = \frac{\sum_{j=1}^m \gamma_{ji} (x_j - \mu_i)(x_j - \mu_i)^T}{\sum_{j=1}^m \gamma_{ji}}$$

Moltiplicatore di Lagrange:

$$\alpha_i = \frac{1}{m} \sum_{j=1}^m \gamma_{ji}$$

Algorithm 5 Mixture of-Gaussian Clustering

Require: Data set $D = \{x_1, x_2, \dots, x_m\}$; Number of Gaussian mixture components k .

```
1: Process:
2: repeat
3:   for  $j = 1, 2, \dots, m$  do
4:     According to (9.30), compute the posterior probabilities that  $x_j$  is generated by each
     Gaussian mixture component, i.e.,  $\gamma_{ji} = p_M(z_j = i \mid x_j) (1 \leq i \leq k)$ 
5:   end for
6:   for  $i = 1, 2, \dots, k$  do
7:     Compute the updated mean vector:  $\mu'_i = \frac{\sum_{j=1}^m \gamma_{ji} x_j}{\sum_{j=1}^m \gamma_{ji}}$ 
8:     Compute the updated covariance matrix:  $\Sigma'_i = \frac{\sum_{j=1}^m \gamma_{ji} (x_j - \mu'_i)(x_j - \mu'_i)^T}{\sum_{j=1}^m \gamma_{ji}}$ 
9:     Compute the updated mixture coefficient:  $\alpha'_i = \frac{1}{m} \sum_{j=1}^m \gamma_{ji}$ 
10:  end for
11:  Update the model parameters:  $\{(\alpha_i, \mu_i, \Sigma_i) \mid 1 \leq i \leq k\}$  to  $\{(\alpha'_i, \mu'_i, \Sigma'_i) \mid 1 \leq i \leq k\}$ 
12: until The termination condition is met
13:  $C_i = \emptyset (1 \leq i \leq k)$ ;
14: for  $j = 1, 2, \dots, m$  do
15:   Determine the cluster label  $\lambda_j$  of  $x_j$  according to (9.31);  $\lambda_j = \arg \max_{i \in \{1, 2, \dots, k\}} \gamma_{ji}$ 
16:   Move  $x_j$  to the corresponding cluster:  $C_{\lambda_j} = C_{\lambda_j} \cup \{x_j\}$ 
17: end for
Ensure: Clusters  $\mathcal{C} = \{C_1, C_2, \dots, C_k\}$ .
```

Risultati del Clustering